侯捷 C++Startup揭密：C++程序的生前和死后

1. 前言，如何自定Startup code

- C++進入點是 main() 嗎？ 生前
- 什麼代碼比 main() 更早被執行？
- 什麼代碼在 main() 結束後才被執行？ 死后
- 為什麼上述代碼可以如此行為？
- Heap 的結構如何？
- I/O 的結構如何？

我們的目標

- 徹底解決上頁所提的每一項疑問

你應具備的基礎

- 是個 C++ programmer

我們所觀察的代碼

VC6, VC10

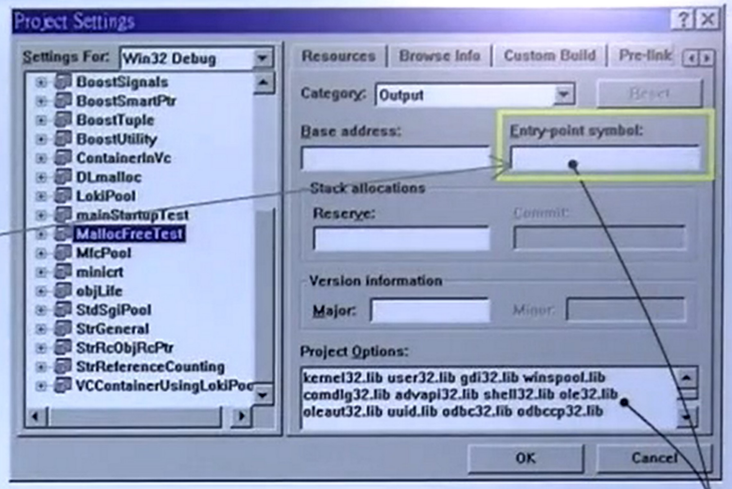
源碼之前了無秘密

Bibliography

- 《程序員的自我修養 — 鏈接、裝載與庫》
  俞甲子 / 石凡 / 潘愛民 著，電子工業出版社

- 《程式設計師的自我修養 — 連結·載入·程式庫》
  俞甲子 / 石凡 / 潘愛民 著，碁峰出版社

自定 Startup code (Entry-Point Symbol)



/ENTRY (Entry-Point Symbol)

The Entry-Point Symbol (/ENTRY:*function*) option sets the **starting address** for an .EXE file or DLL. (To find this option in the development environment, click **Settings** on the **Project** menu. Then click the **Link** tab, and click **Output** in the **Category** box.)

Type a function name in the **Entry-Point Symbol** text box (or in the *function* argument on the command line). The function must be defined with the **__stdcall** calling convention. The parameters and return value must be defined as documented in the Win32 API for **WinMain** (for an .EXE file) or **DllEntryPoint** (for a DLL). It is recommended that you let the linker set the entry point so that the C run-time library is initialized correctly, and C++ constructors for static objects are executed.

CRT 這兩件就是 startup code 的主要用途

By default, the starting address is a function name from the C run-time library. The linker selects it according to the attributes of the program, as shown in the following table.

	Function name	Default for
1	**mainCRTStartup** (or **wmainCRTStartup**)	An application using /SUBSYSTEM:**CONSOLE**; calls **main** (or **wmain**)
2	**WinMainCRTStartup** (or **wWinMainCRTStartup**)	An application using /SUBSYSTEM:**WINDOWS**; calls **WinMain** (or **wWinMain**), which must be defined with **__stdcall**
3	**_DllMainCRTStartup**	A DLL; calls **DllMain**, which must be defined with **__stdcall**, if it exists

Startup code 的三個可能形式 (函數名稱)

完成之後 Project Options 會多出 **/entry:"*MyStartup*"**

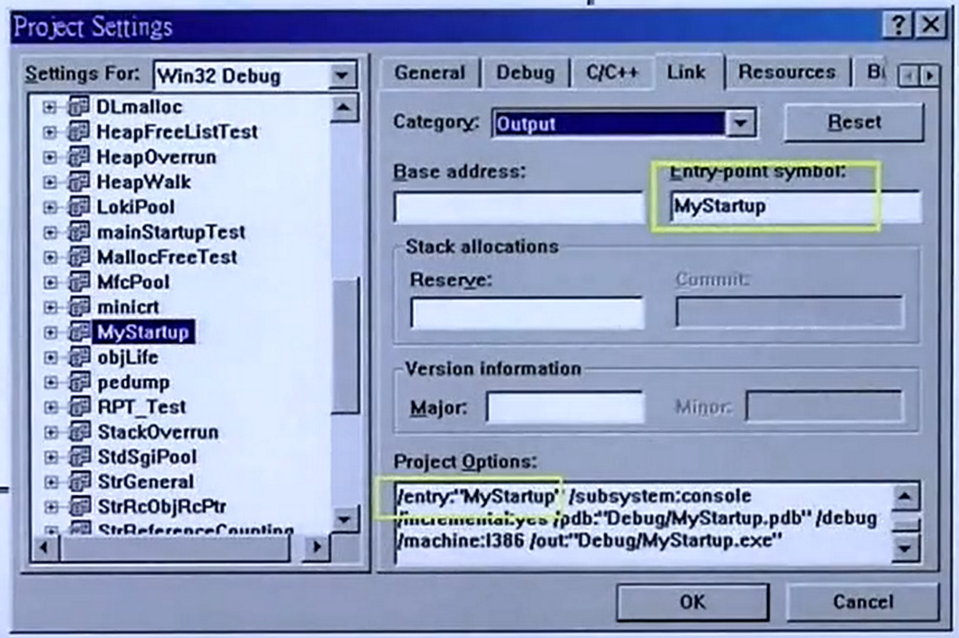
Boolan 博览

自定 Startup code

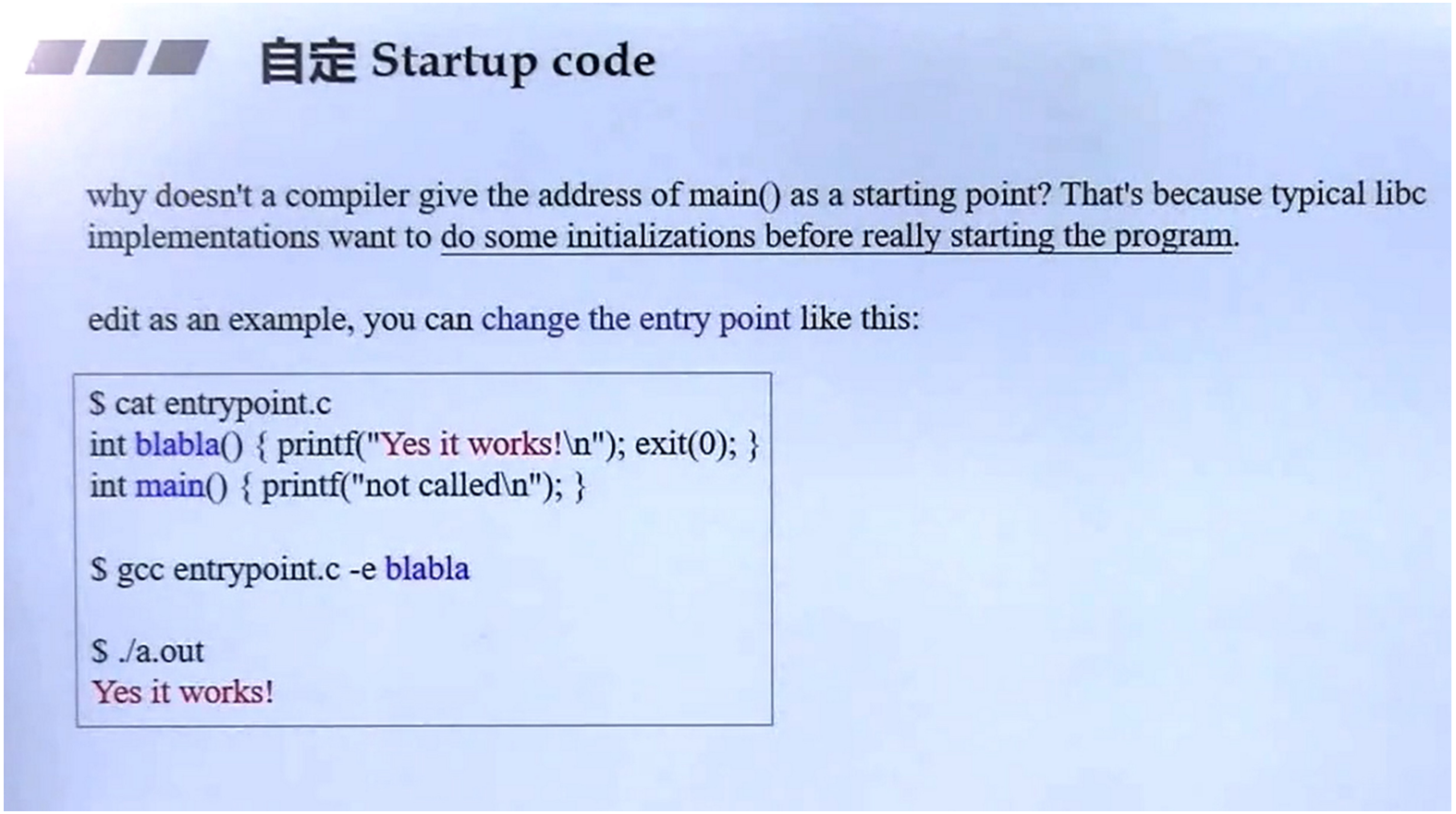
```
#include <windows.h>
int MyStartup( void )
{
 int a=10;
 HANDLE crtHeap=HeapCreate(HEAP_NO_SERIALIZE, 0x010, 4000*1024);
 int *p=(int*)HeapAlloc(crtHeap, HEAP_ZERO_MEMORY, 0x010);
 int i,j;

 for(i=0; i<100; i++)
 {
     //int j;
     for(j=0; j<100; j++,p++)
     {
        *p = i*100 + (j+1);
     }
 }

 MessageBoxA(NULL,p,"abcd",MB_OK);
 return 0;
}
```

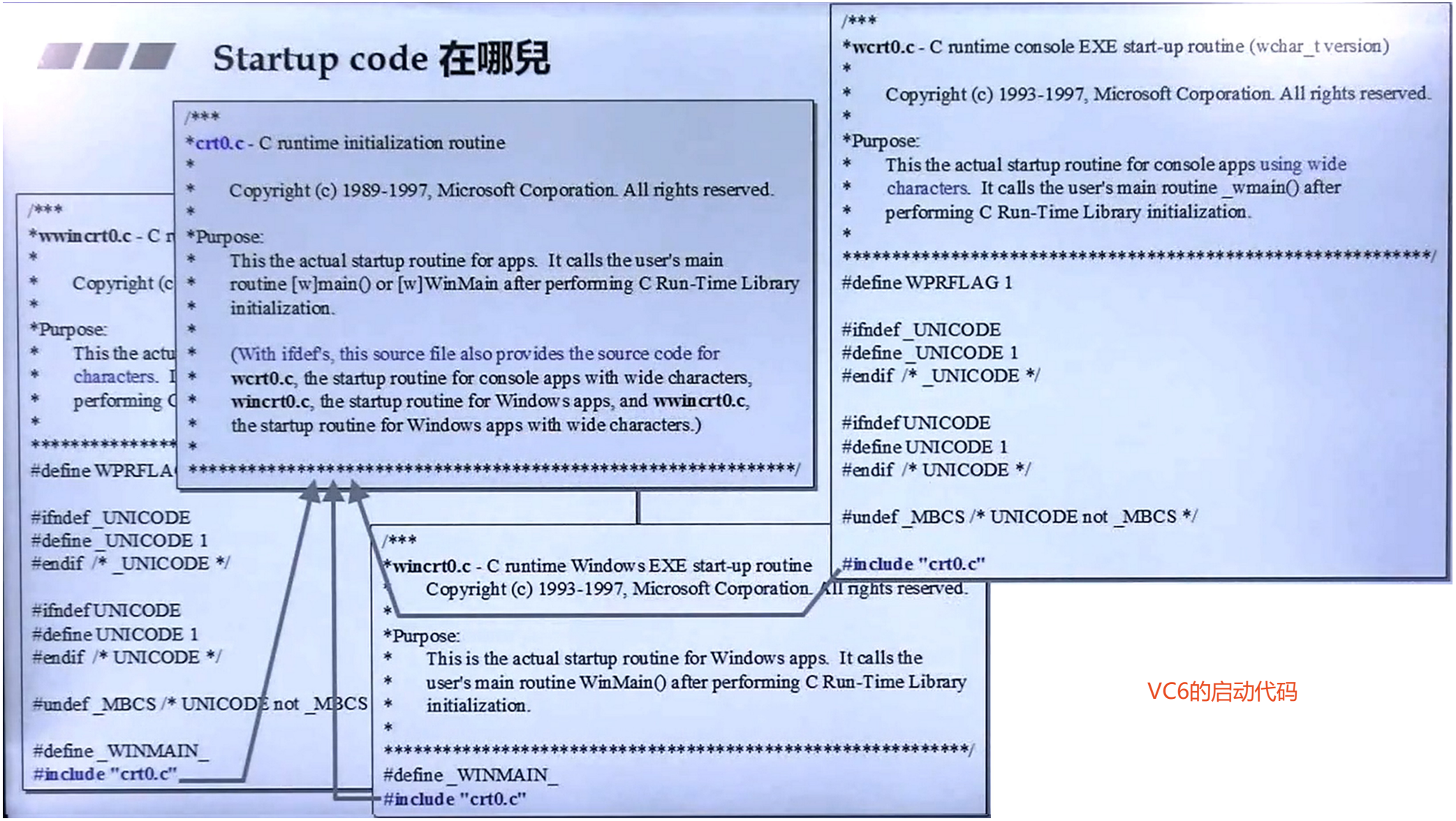


main由Startup code调用

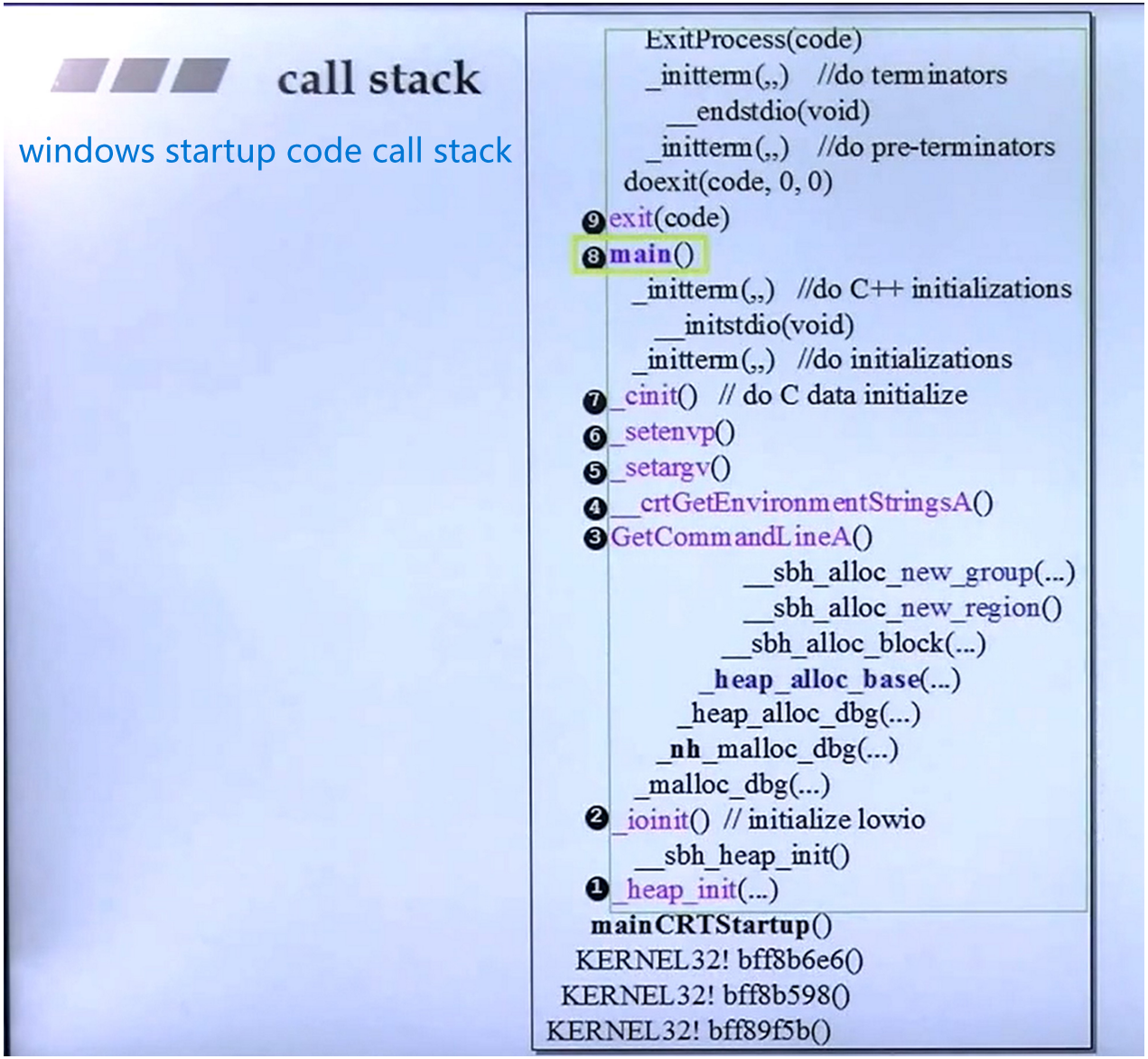


2. 默认的Startup code在哪儿

Startup code 在哪兒



VC6的启动代码



以 _heap_init 为例，它不只包含 _sbh_heap_init()，还包含其它子调用，只不过我们在这里只关心 _sbh_heap_init()，因此只展示出来这个子调用。其它调用也是同理。

3. Startup code代码摘要

mainCRTStartup()

mainCRTStartup() in crt0.c

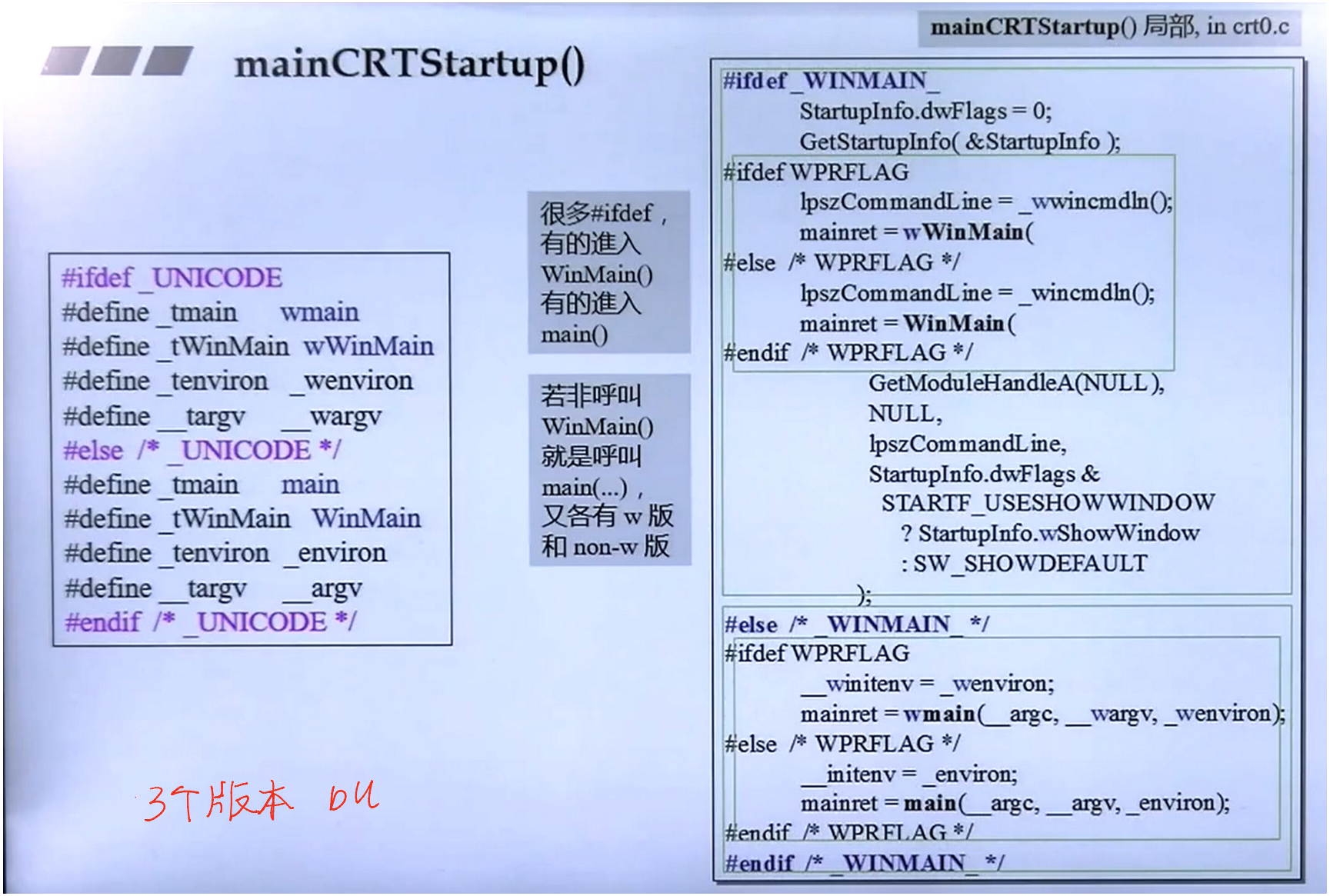
VC6

這是整理後的結果。實際上有很多#ifdef，有的進入 WinMain() 有的進入 main()

```
01 void mainCRTStartup(void)
02 {
03     int mainret;
04     // Get the full Win32 version
05     _osver = GetVersion();
06     _winminor = (_osver >> 8) & 0x00FF ;
07     _winmajor = _osver & 0x00FF ;
08     _winver = (_winmajor << 8) + _winminor;
09     _osver = (_osver >> 16) & 0x00FFFF ;
10
11     1 if ( !_heap_init(0) )          /* initialize heap */
12         fast_error_exit(_RT_HEAPINIT);  /* write message and die */
13
```

```
14     /*
15      * Guard the remainder of the initialization code and the call
16      * to user's main, or WinMain, function in a __try/__except
17      * statement.
18      */
19     __try {
20   2 _ioinit();                  /* initialize lowio */
21     /* get cmd line info */
22   3 _acmdln = (char *)GetCommandLineA();
23     /* get environ info */
24   4 _aenvptr = (char *)__crtGetEnvironmentStringsA();
25   5 _setargv();                 字不錯处理
26   6 _setenvp();
27   7 _cinit();      /* do C data initialize */
28   8 __initenv = _environ;
29   8 mainret = main(__argc, __argv, _environ);
30   9 exit(mainret);                ^——>注意 calling convention
31     }
32     __except ( _XcptFilter(GetExceptionCode(),
33                            GetExceptionInformation()) )
34     {
35       // Should never reach here
36       _exit( GetExceptionCode() );
37     } /* end of try - except */
38 }
```

Boolan 博览



4. Heap_init() - Startup的首要工程

VC6 _heap_init()

```
    ExitProcess(code)
    _initterm(,,)   //do terminators
      __endstdio(void)
    _initterm(,,)   //do pre-terminators
  doexit(code, 0, 0)
9 exit(code)
8 main()
    _initterm(,,)   //do C++ initializations
      __initstdio(void)
    _initterm(,,)   //do initializations
7 _cinit()  // do C data initialize
6 _setenvp()
5 _setargv()
4 __crtGetEnvironmentStringsA()
3 GetCommandLineA()
          __sbh_alloc_new_group(...)
          __sbh_alloc_new_region()
           sbh_alloc_block(...)
        heap_alloc_base(...)
      _heap_alloc_dbg(...)
    _nh_malloc_dbg(...)
  _malloc_dbg(...)
2 _ioinit()  // initialize lowio
    __sbh_heap_init()
1 _heap_init(...)
  mainCRTStartup()
KERNEL32! bff8b6e6()
KERNEL32! bff8b598()
KERNEL32! bff89f5b()
```

我的《C++內存管理機制》(全五講) 中的第三講 malloc/free 對於 CRT 的 SBH 設計和運行有極詳細的說明。本課程此處之 _heap_init() 及其引發的 memory 分配與管理, 是屬相同主題。

本處之特別：

1, 藉 Startup code 處理字符串 (包括命令行參數, 環境變數) 過程中的詳細數據觀察，釐清、確認和加強所有環節。

2, 增加說明 Windows Heap (操作系統層) 的結構和管理。(當 CRT's SBH 不再存在時)

SBH: Small Block Heap

何謂 small ?

內存塊

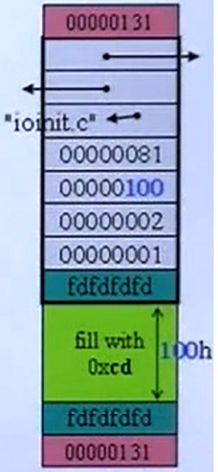
1, 從何處來 ?

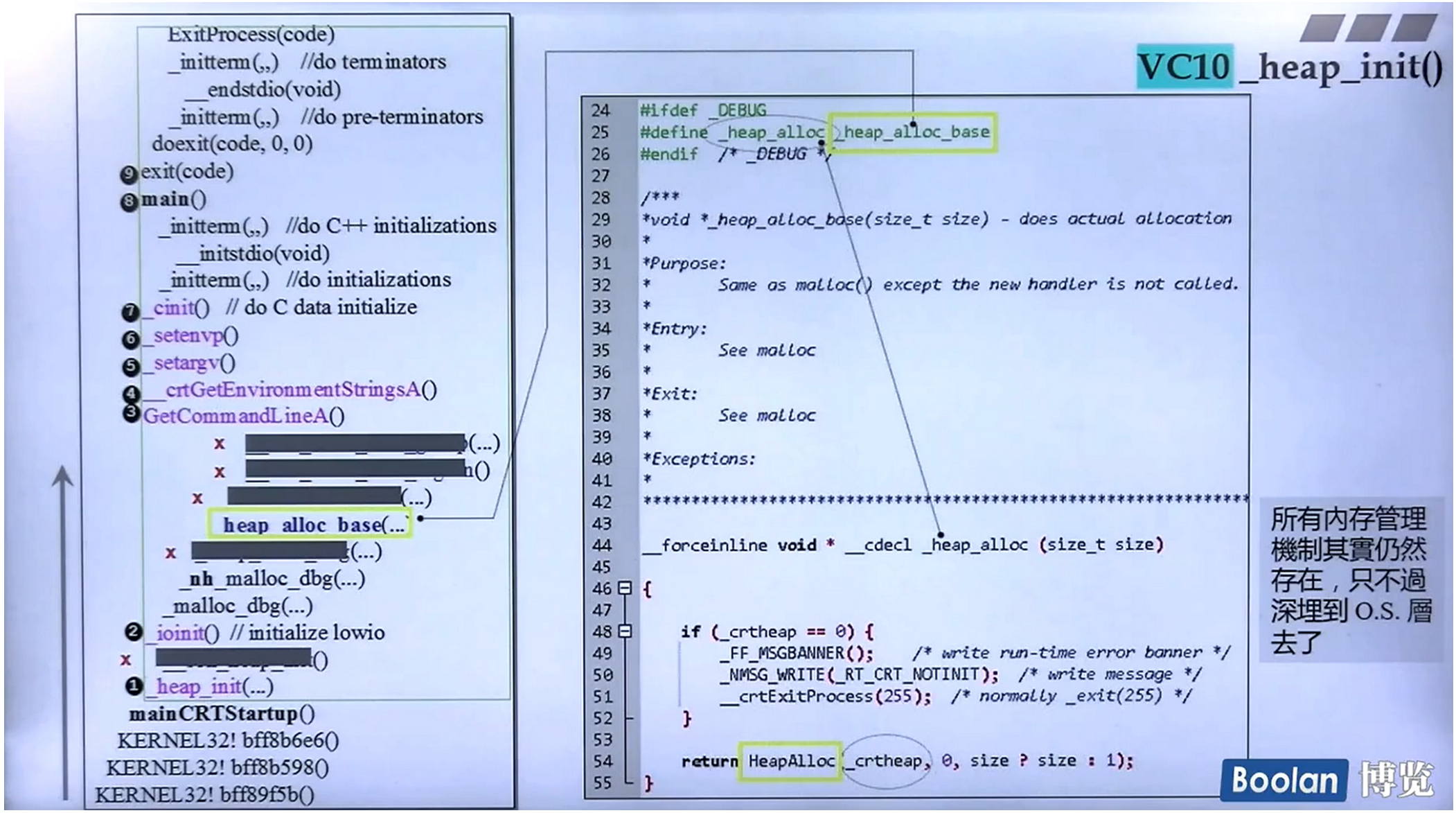
2, 大小幾何 ?

3, 回收至何處 ?

```
if (size <= __sbh_threshold) {    //3F8, i.e. 1016
    pvReturn = __sbh_alloc_block(size);
    if (pvReturn)  return pvReturn;
}
if (size == 0)  size = 1;
size = (size + ...) & ~(...);  对齐
return HeapAlloc(_crtheap, 0, size);  交由操作系统分配
```

DEBUG 模式下:



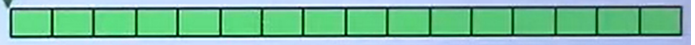


无论大小，直接交付 OS 管理，但它的行为其实和 VC6 中 SBH 的行为一样。

SBH 之始 – _heap_init() 和 __sbh_heap_init()

CRT 會先為自己建立一個 _crtheap，然後從中分配 SBH 所需的 headers, regions 做為管理之用。App. 動態分配時若 size > threshold 就調用 HeapAlloc() 從 _crtheap 取。若 size <= threshold 就徑自從 SBH 取 (實際區塊來自 VirtualAlloc())

16 個 Headers



heapinit.c

```
int __cdecl _heap_init (
        int mtflag
        )
{       分配所得 _crtheap 用于管理 SBH
        // Initialize the "big-block" heap first.
        if ( (_crtheap = HeapCreate( mtflag ? 0 : HEAP_NO_SERIALIZE,
                                     BYTES_PER_PAGE, 0 )) == NULL )
                return 0;                                  4096

        // Initialize the small-block heap
        if (__sbh_heap_init() == 0)
        {
            HeapDestroy(_crtheap);
            return 0;
        }
        return 1;
}
```

不論 big-block heap or small-block heap，只要失敗就 return 0，別玩了。

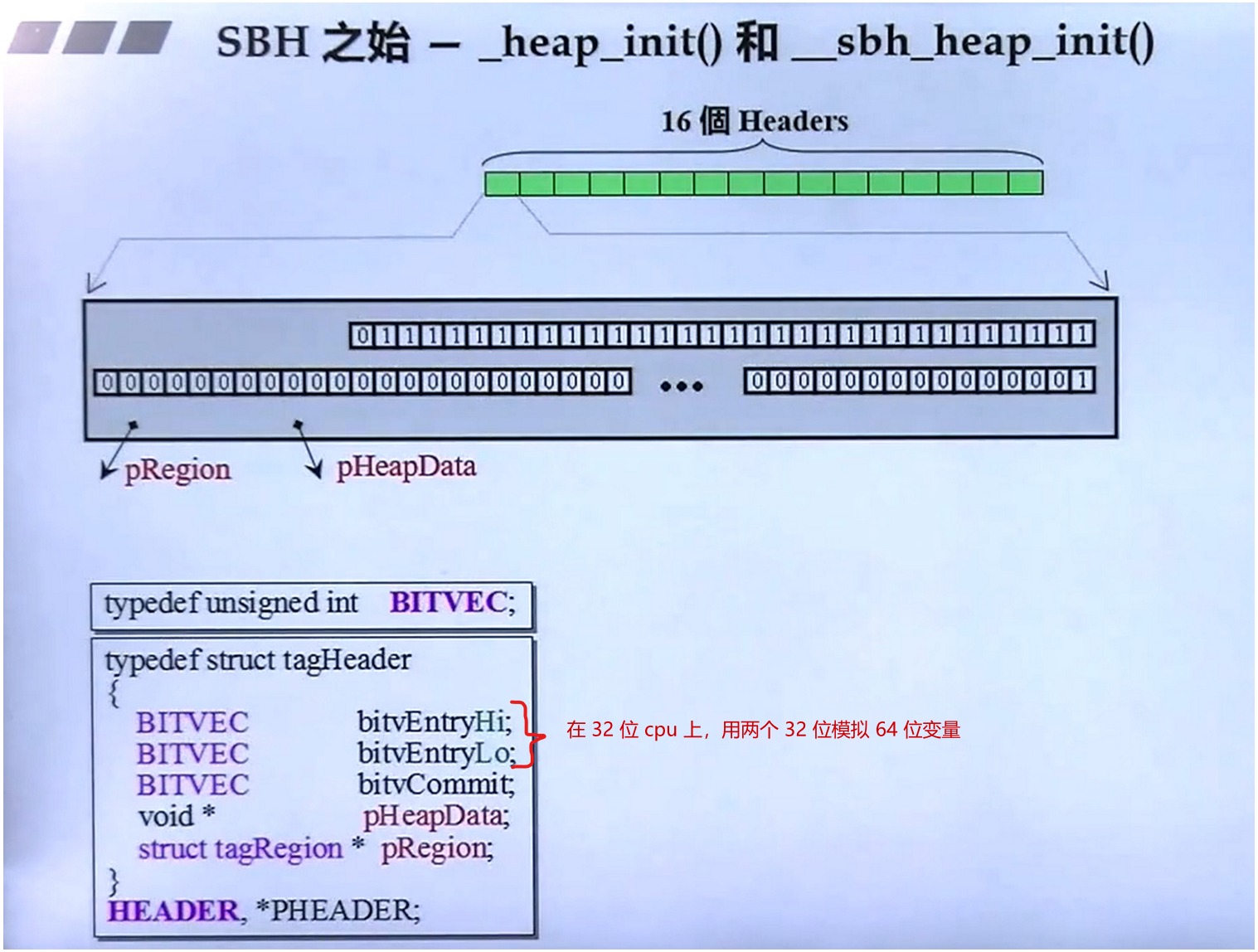
sbheap.c

```
int __cdecl __sbh_heap_init (void)
{
    if (!(__sbh_pHeaderList =
            HeapAlloc(_crtheap, 0, 16 * sizeof(HEADER))))
        return FALSE;

    __sbh_pHeaderScan = __sbh_pHeaderList;
    __sbh_pHeaderDefer = NULL;
    __sbh_cntHeaderList = 0;
    __sbh_sizeHeaderList = 16;

    return TRUE;
}
```


Boolan 博览

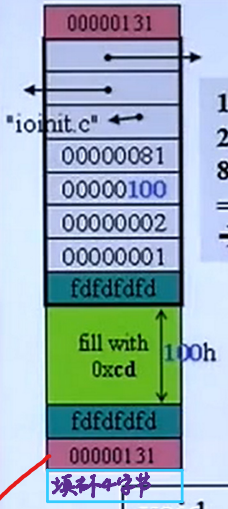

SBH 之始 – _heap_init() 和 __sbh_heap_init()
16 個 Headers
pRegion
pHeapData
typedef unsigned int BITVEC;
typedef struct tagHeader
{
BITVEC bitvEntryHi;
BITVEC bitvEntryLo;
BITVEC bitvCommit;
void * pHeapData;
struct tagRegion * pRegion;
}
HEADER, *PHEADER;
在 32 位 cpu 上，用两个 32 位模拟 64 位变量

5. 内存分配精解1

VC6 内存分配

```
ExitProcess(code)
  _initterm(,,)   //do terminators
    __endstdio(void)
  _initterm(,,)   //do pre-terminators
 doexit(code, 0, 0)
9 exit(code)
8 main()
  _initterm(,,)   //do C++ initializations
    __initstdio(void)
  _initterm(,,)   //do initializations
7 _cinit()  // do C data initialize
6 _setenvp()
5 _setargv()
4 __crtGetEnvironmentStringsA()
3 GetCommandLineA()
        __sbh_alloc_new_group(...)
      __sbh_alloc_new_region()
    __sbh_alloc_block(...)
  _heap_alloc_base(...)
 _heap_alloc_dbg(...)
_nh_malloc_dbg(...)
 malloc_dbg(...)
2 ioinit()  // initialize lowio
  __sbh_heap_init()
1 _heap_init(...)
 mainCRTStartup()
KERNEL32! bff8b6e6()
KERNEL32! bff8b598()
KERNEL32! bff89f5b()
```

```
/* Memory block identification */
#define _FREE_BLOCK      0
#define _NORMAL_BLOCK    1
#define _CRT_BLOCK       2
#define _IGNORE_BLOCK    3
#define _CLIENT_BLOCK    4
#define _MAX_BLOCKS      5
```



100h + data
24h + debug
8h cookie
= 12ch align
➔ 130h eventually size

```
#ifndef _DEBUG
#define _malloc_crt   malloc
...
#else  /* _DEBUG */
#define _THISFILE     __FILE__
#define _malloc_crt(s)   _malloc_dbg
                           (s, _CRT_BLOCK,
                            _THISFILE, __LINE__)
...     分配内存并且登记 DEBUG 信息（文件名、行号）
#endif  /* _DEBUG */
```

```
typedef struct {
    long osfhnd;
    char osfile;
    char pipech;
} ioinfo;
```

```
void __cdecl _ioinit (void)
{
...
line81  if ( (pio = _malloc_crt( IOINFO_ARRAY_ELTS * sizeof(ioinfo) ))
              == NULL )
...
}
```

32

8

第1次分配内存大小256

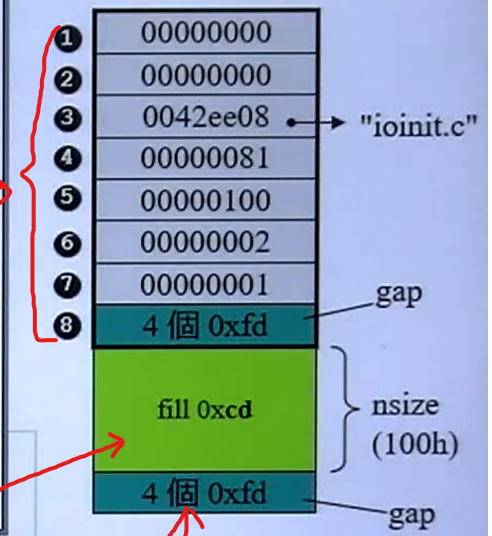
最终所得内存块大小为 130h，但需要用最后一个字节表示该内存块是否已经被分配，因此记录 131

Boolan 博览

VC6 內存分配

```
ExitProcess(code)
_initterm(,,)   //do terminators
__endstdio(void)
_initterm(,,)   //do pre-terminators
doexit(code, 0, 0)
9 exit(code)
8 main()
_initterm(,,)   //do C++ initializations
__initstdio(void)
_initterm(,,)   //do initializations
7 _cinit()  // do C data initialize
6 _setenvp()
5 _setargv()
4 __crtGetEnvironmentStringsA()
3 GetCommandLineA()
__sbh_alloc_new_group(...)
__sbh_alloc_new_region()
__sbh_alloc_block(...)
heap_alloc_base(...)
heap_alloc_dbg(...)
_nh_malloc_dbg(...)
_malloc_dbg(...)
2 _ioinit()  // initialize lowio
__sbh_heap_init()
1 _heap_init(...)
mainCRTStartup()
KERNEL32! bff8b6e6()
KERNEL32! bff8b598()
KERNEL32! bff89f5b()
```

```
#define nNoMansLandSize 4
typedef struct _CrtMemBlockHeader
{
    struct _CrtMemBlockHeader * pBlockHeaderNext;
    struct _CrtMemBlockHeader * pBlockHeaderPrev;
    char *          szFileName;
    int             nLine;
    size_t          nDataSize;
    int             nBlockUse;
    long            lRequest;
    unsigned char   gap[nNoMansLandSize];
    /* followed by:
     *  unsigned char  data[nDataSize];
     *  unsigned char  anotherGap[nNoMansLandSize];
     */
} _CrtMemBlockHeader;
```




```
…
blockSize = sizeof(_CrtMemBlockHeader) + nSize + nNoMansLandSize;   ➔ 調整大小
…
pHead = (_CrtMemBlockHeader *)_heap_alloc_base(blockSize);   ➔ 從 SBH 擷取一內存塊
… (續下頁)   ➔ 設妥 Debug Heap
```

Boolan 博览

VC6 内存分配

```
ExitProcess(code)
  _initterm(,,)   //do terminators
    __endstdio(void)
  _initterm(,,)   //do pre-terminators
doexit(code, 0, 0)
9 exit(code)
8 main()
  _initterm(,,)   //do C++ initializations
    __initstdio(void)
  _initterm(,,)   //do initializations
7 _cinit()  // do C data initialize
6 _setenvp()
5 _setargv()
4 __crtGetEnvironmentStringsA()
3 GetCommandLineA()
                    __sbh_alloc_new_group(...)
                    __sbh_alloc_new_region()
                    __sbh_alloc_block(...)
                  heap_alloc_base(...)
                heap_alloc_dbg(...)
              _nh_malloc_dbg(...)
            _malloc_dbg(...)
2 _ioinit()  // initialize lowio
    __sbh_heap_init()
1 _heap_init(...)
  mainCRTStartup()
KERNEL32! bff8b6e6()
KERNEL32! bff8b598()
KERNEL32! bff89f5b()
```

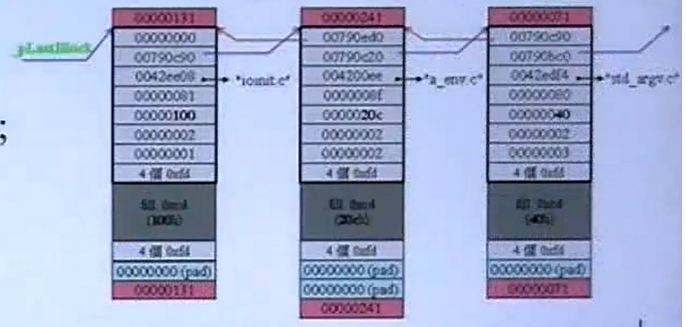
```
… (承上頁)
if ( _pFirstBlock)
    _pFirstBlock->pBlockHeaderPrev = pHead;
else
    _pLastBlock = pHead;

pHead->pBlockHeaderNext =  _pFirstBlock;
pHead->pBlockHeaderPrev = NULL;
pHead->szFileName = (char*)szFileName;
pHead->nLine = nLine;
pHead->nDataSize = nSize;
pHead->nBlockUse = nBlockUse;
pHead->lRequest = lRequest;

/* link blocks together */
_pFirstBlock = pHead;

/* fill in gap before and after real block */
memset((void *)pHead->gap, _bNoMansLandFill, nNoMansLandSize);
memset((void *)(pbData(pHead) + nSize), _bNoMansLandFill, nNoMansLandSize);
/* fill data with silly value (but non-zero) */
memset((void *)pbData(pHead), _bCleanLandFill, nSize);
return (void *)pbData(pHead);
```

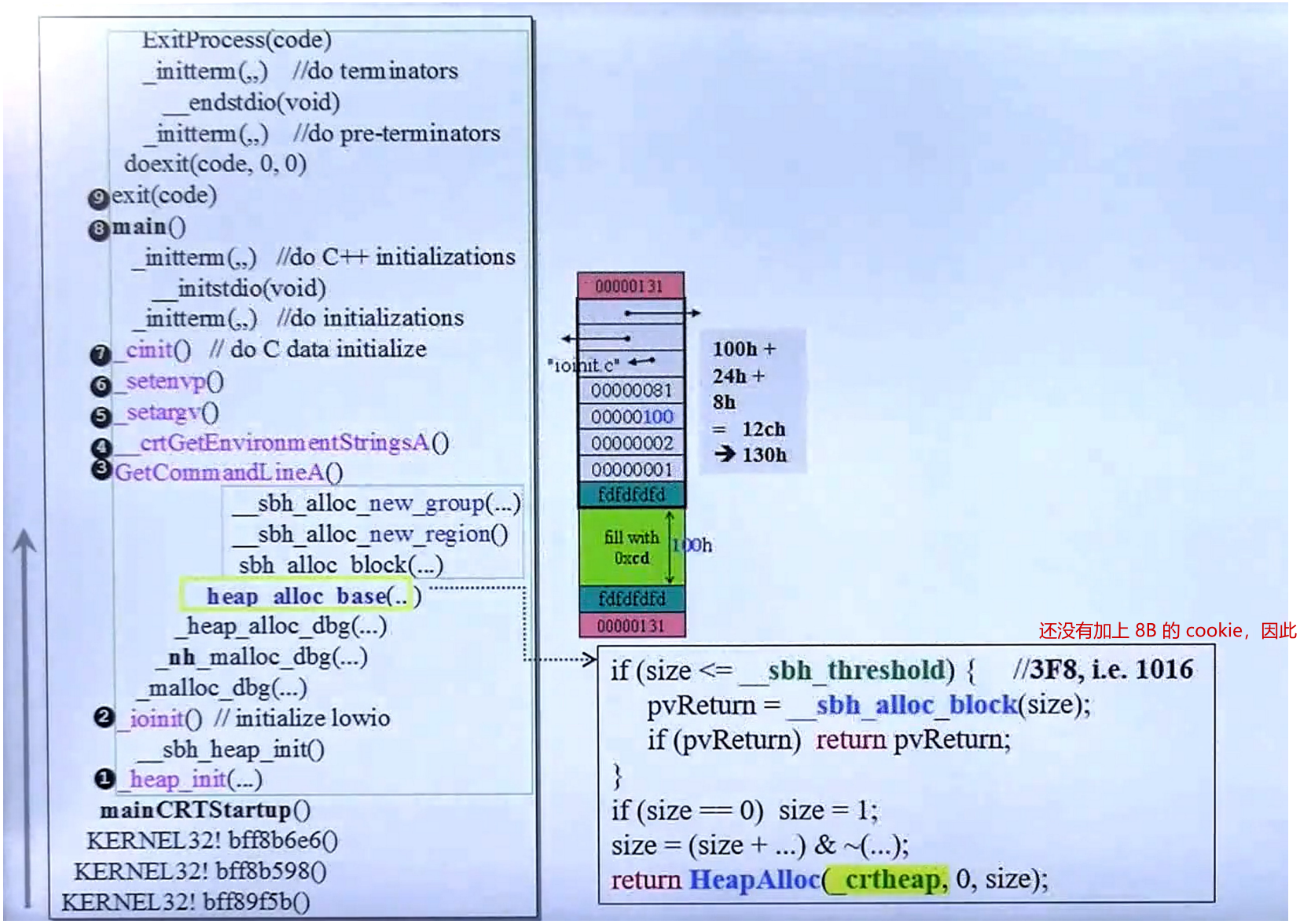
Debug Heap



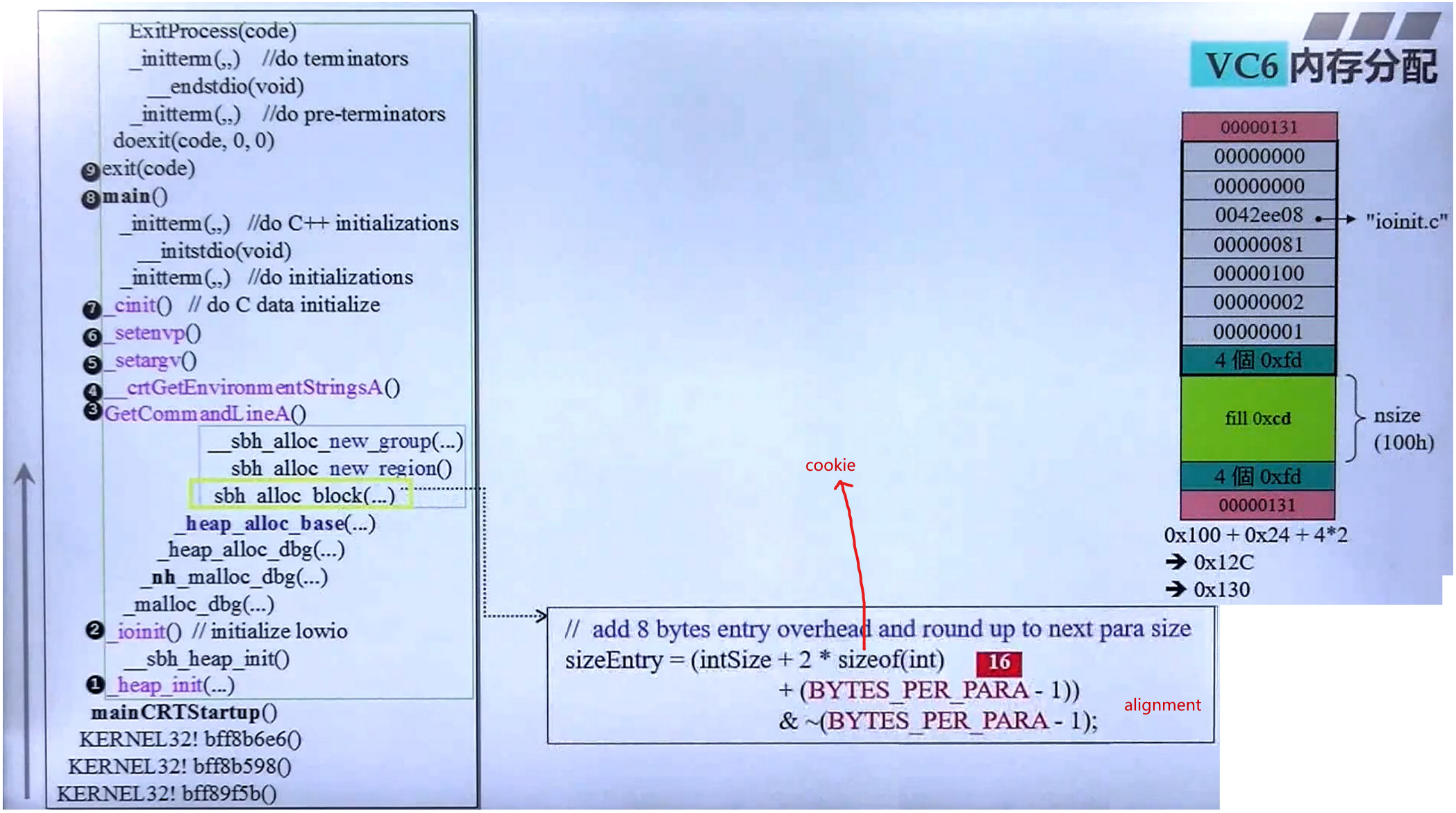

```
static unsigned char _bNoMansLandFill = 0xFD;
static unsigned char _bDeadLandFill   = 0xDD;
static unsigned char _bCleanLandFill  = 0xCD;

static _CrtMemBlockHeader * _pFirstBlock;
static _CrtMemBlockHeader * _pLastBlock;
```

Boolan 博览

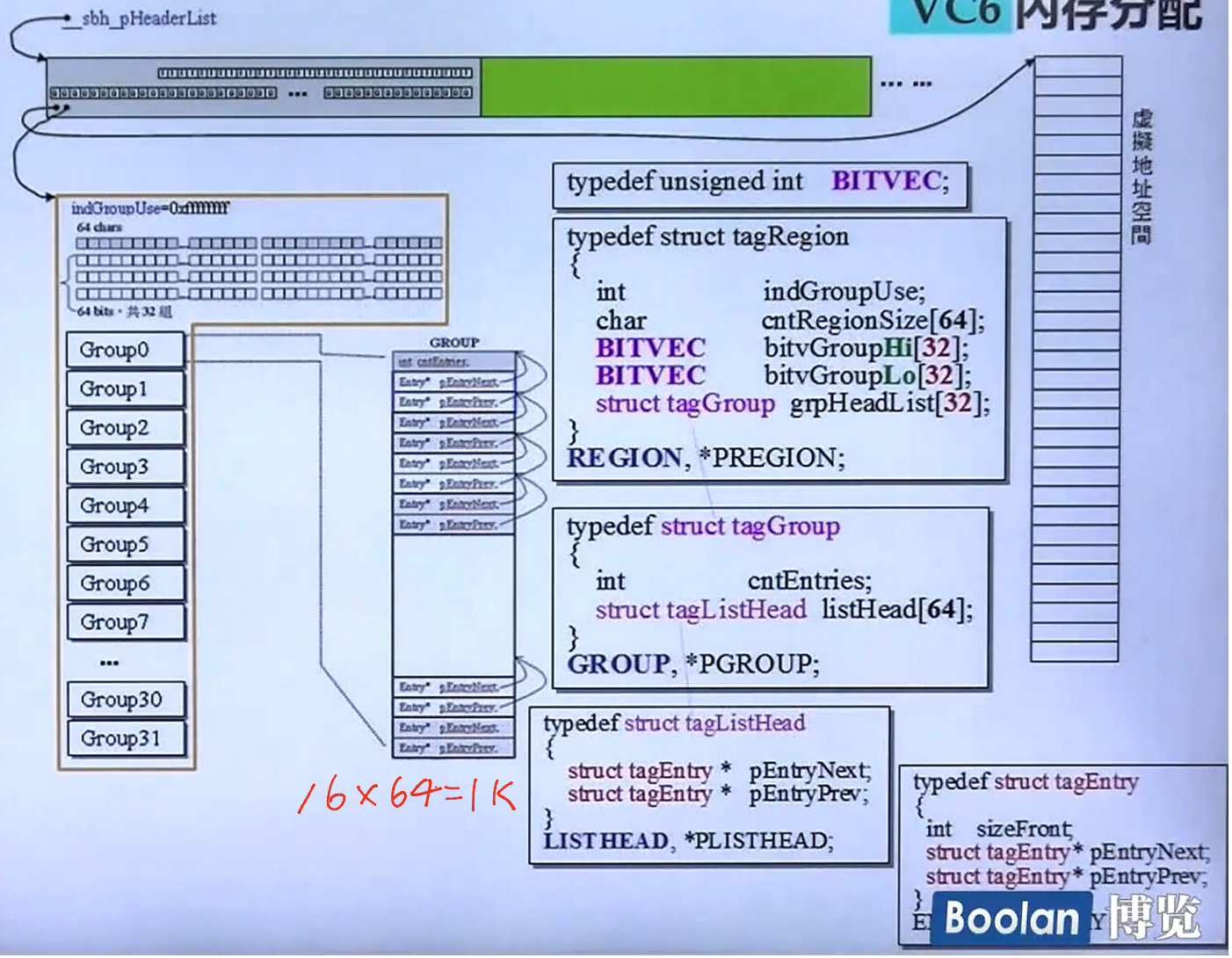

ExitProcess(code)
_initterm(,,) //do terminators
__endstdio(void)
_initterm(,,) //do pre-terminators
doexit(code, 0, 0)
9 exit(code)
8 main()
_initterm(,,) //do C++ initializations
__initstdio(void)
_initterm(,,) //do initializations
7 _cinit() // do C data initialize
6 _setenvp()
5 _setargv()
4 __crtGetEnvironmentStringsA()
3 GetCommandLineA()
__sbh_alloc_new_group(...)
__sbh_alloc_new_region()
sbh_alloc_block(...)
heap_alloc_base(..)
_heap_alloc_dbg(...)
_nh_malloc_dbg(...)
_malloc_dbg(...)
2 _ioinit() // initialize lowio
__sbh_heap_init()
1 _heap_init(...)
mainCRTStartup()
KERNEL32! bff8b6e6()
KERNEL32! bff8b598()
KERNEL32! bff89f5b()
00000131
"ioinit.c"
00000081
00000100
00000002
00000001
fdfdfdfd
fill with 0xcd
100h
fdfdfdfd
00000131
100h +
24h +
8h
= 12ch
➔ 130h
还没有加上 8B 的 cookie，因此 size=1016
if (size <= __sbh_threshold) { //3F8, i.e. 1016
pvReturn = __sbh_alloc_block(size);
if (pvReturn) return pvReturn;
}
if (size == 0) size = 1;
size = (size + ...) & ~(...);
return HeapAlloc(_crtheap, 0, size);

6. 内存分配精解2


ExitProcess(code)
_initterm(,,) //do terminators
__endstdio(void)
_initterm(,,) //do pre-terminators
doexit(code, 0, 0)
9 exit(code)
8 main()
_initterm(,,) //do C++ initializations
__initstdio(void)
_initterm(,,) //do initializations
7 _cinit() // do C data initialize
6 _setenvp()
5 _setargv()
4 __crtGetEnvironmentStringsA()
3 GetCommandLineA()
__sbh_alloc_new_group(...)
sbh_alloc_new_region()
sbh_alloc_block(...)
_heap_alloc_base(...)
_heap_alloc_dbg(...)
_nh_malloc_dbg(...)
_malloc_dbg(...)
2 _ioinit() // initialize lowio
__sbh_heap_init()
1 _heap_init(...)
mainCRTStartup()
KERNEL32! bff8b6e6()
KERNEL32! bff8b598()
KERNEL32! bff89f5b()
cookie
// add 8 bytes entry overhead and round up to next para size
sizeEntry = (intSize + 2 * sizeof(int) 16
+ (BYTES_PER_PARA - 1))
& ~(BYTES_PER_PARA - 1);
alignment
VC6 内存分配
00000131
00000000
00000000
0042ee08
"ioinit.c"
00000081
00000100
00000002
00000001
4 個 0xfd
fill 0xcd
nsize
(100h)
4 個 0xfd
00000131
0x100 + 0x24 + 4*2
0x12C
0x130

VC6 内存分配

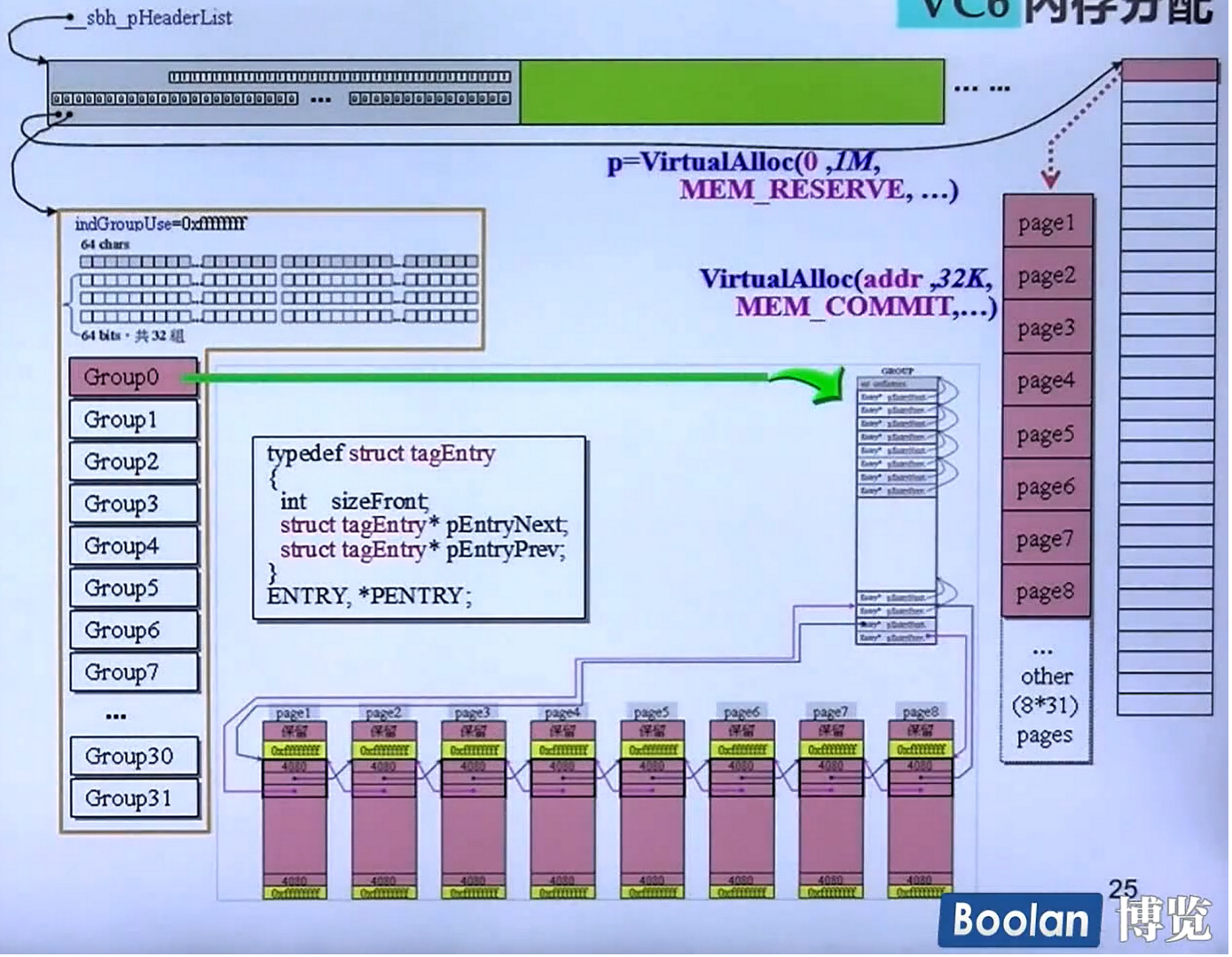
```
ExitProcess(code)
  _initterm(,,)   //do terminators
    __endstdio(void)
  _initterm(,,)   //do pre-terminators
doexit(code, 0, 0)
9 exit(code)
8 main()
  _initterm(,,)   //do C++ initializations
    __initstdio(void)
  _initterm(,,)   //do initializations
7 _cinit()  // do C data initialize
6 _setenvp()
5 _setargv()
4 __crtGetEnvironmentStringsA()
3 GetCommandLineA()
                    sbh_alloc_new_group(...)
                    sbh_alloc_new_region()
                  __sbh_alloc_block(...)
                _heap_alloc_base(...)
              _heap_alloc_dbg(...)
            _nh_malloc_dbg(...)
          _malloc_dbg(...)
2 _ioinit()  // initialize lowio
    __sbh_heap_init()
1 _heap_init(...)
  mainCRTStartup()
KERNEL32! bff8b6e6()
KERNEL32! bff8b598()
KERNEL32! bff89f5b()
```



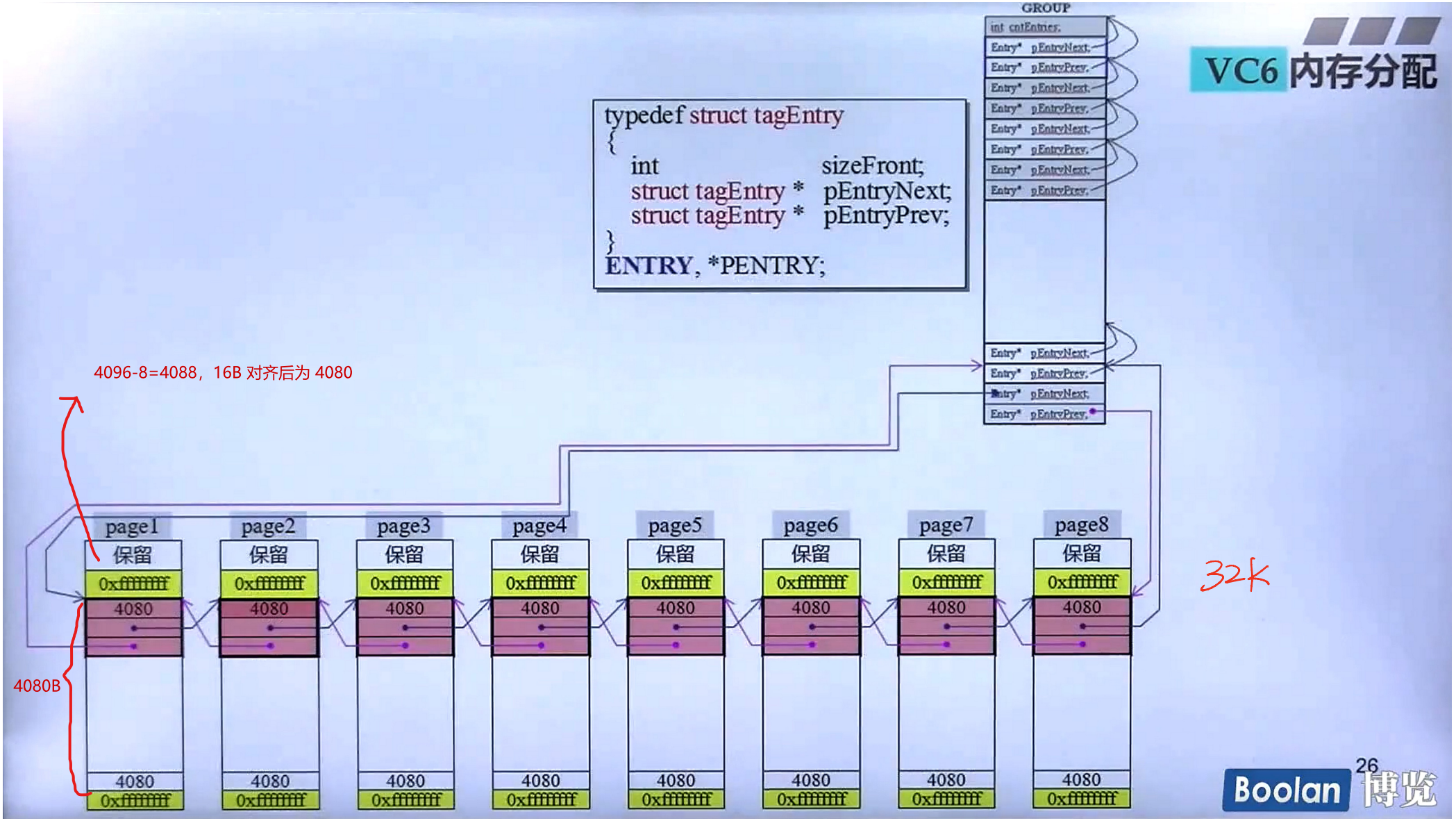
Boolan 博览

VC6 内存分配

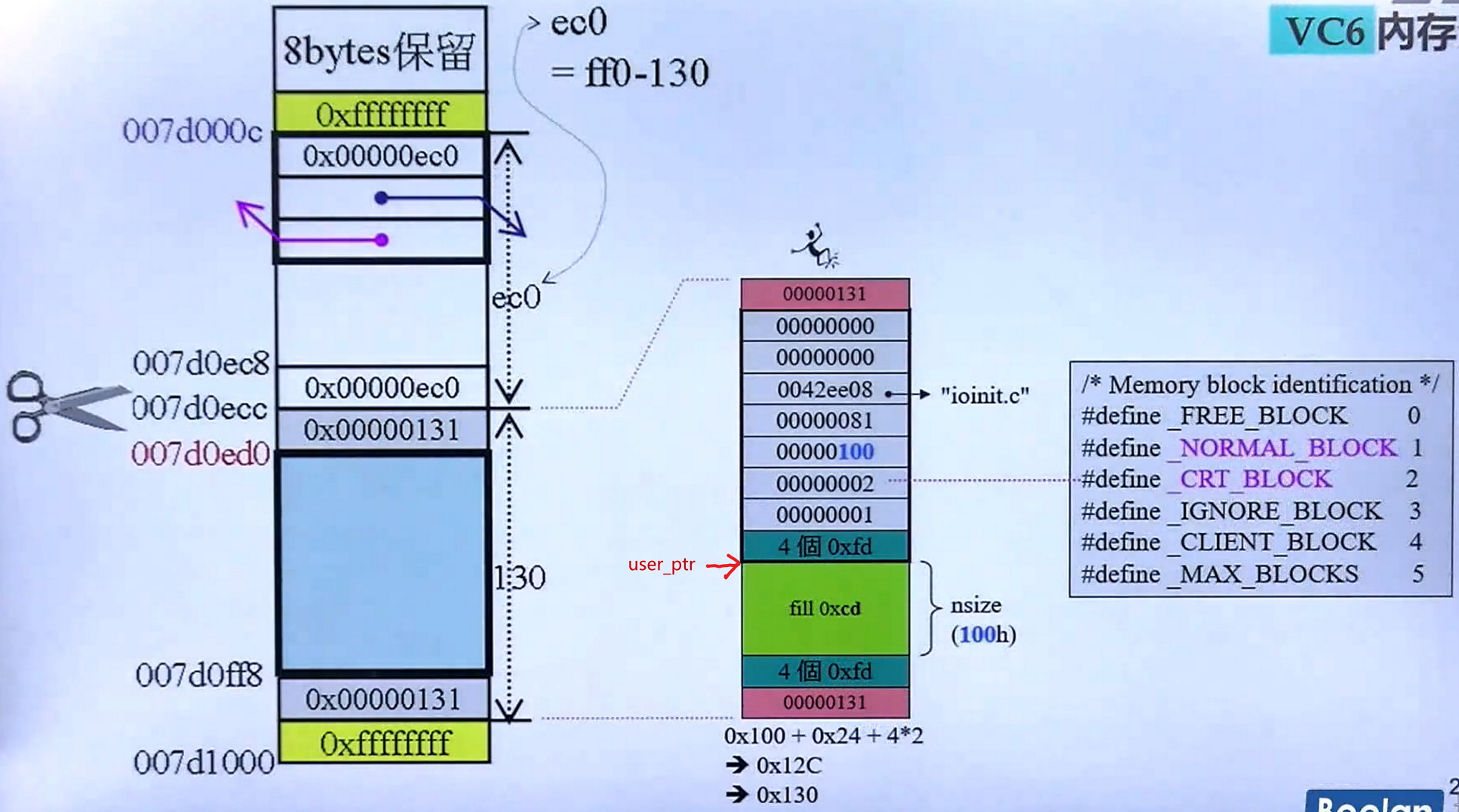
```
ExitProcess(code)
  _initterm(,,)   //do terminators
    __endstdio(void)
  _initterm(,,)   //do pre-terminators
 doexit(code, 0, 0)
9 exit(code)
8 main()
  _initterm(,,)   //do C++ initializations
    __initstdio(void)
  _initterm(,,)   //do initializations
7 _cinit()  // do C data initialize
6 _setenvp()
5 _setargv()
4 __crtGetEnvironmentStringsA()
3 GetCommandLineA()
            __sbh_alloc_new_group(...)
          __sbh_alloc_new_region()
        __sbh_alloc_block(...)
      _heap_alloc_base(...)
    _heap_alloc_dbg(...)
  _nh_malloc_dbg(...)
 _malloc_dbg(...)
2 _ioinit()  // initialize lowio
    __sbh_heap_init()
1 _heap_init(...)
 mainCRTStartup()
KERNEL32! bff8b6e6()
KERNEL32! bff8b598()
KERNEL32! bff89f5b()
```



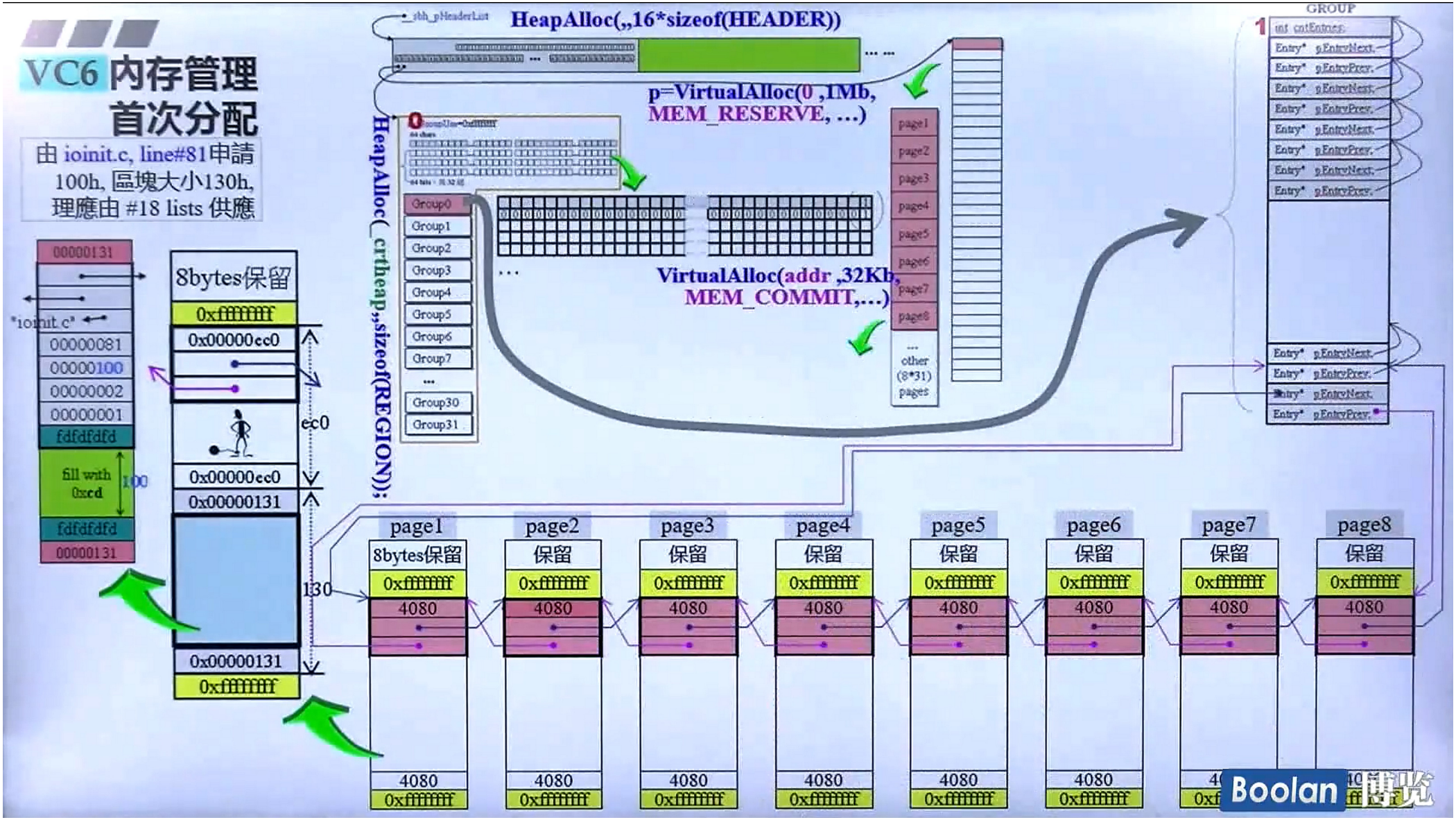
Boolan 博览

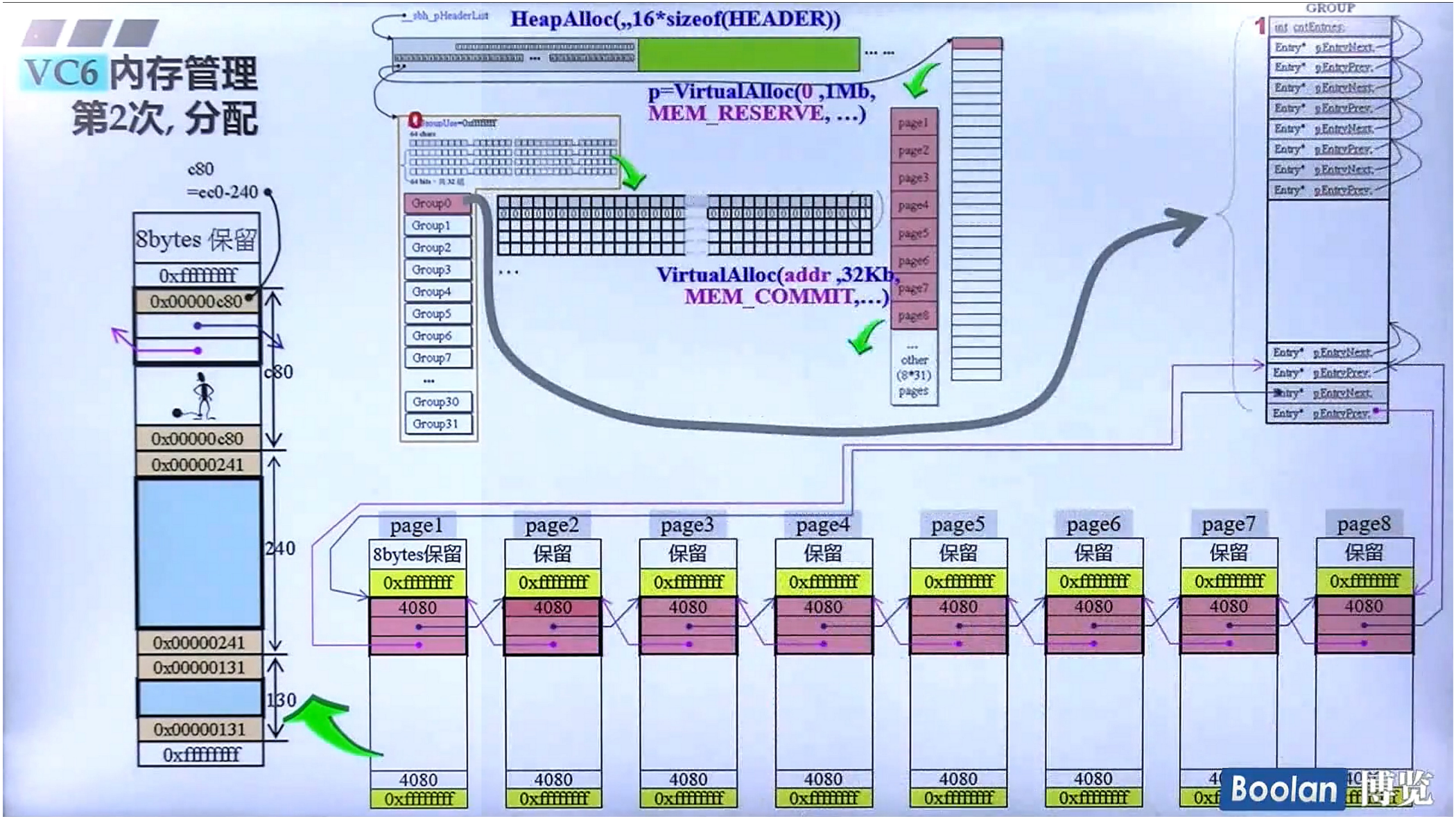

VC6 内存分配
GROUP
int cntEntries;
Entry* pEntryNext;
Entry* pEntryPrev;
typedef struct tagEntry
{
int sizeFront;
struct tagEntry * pEntryNext;
struct tagEntry * pEntryPrev;
}
ENTRY, *PENTRY;
4096-8=4088，16B 对齐后为 4080
page1
page2
page3
page4
page5
page6
page7
page8
保留
0xffffffff
4080
4080B
32K
Boolan 博览
26

7. 内存分配精解3

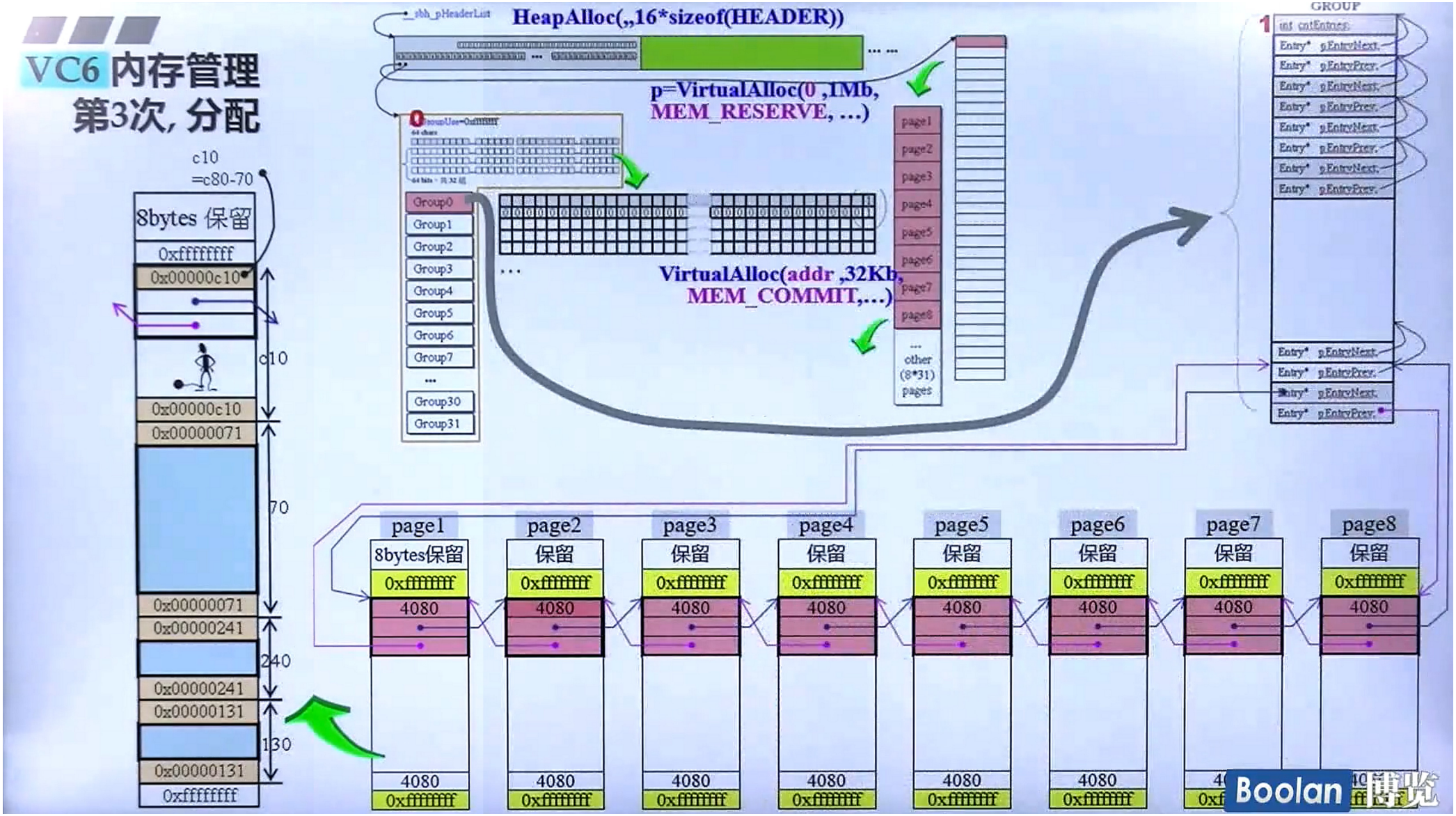

8bytes保留
0xffffffff
0x00000ec0
0x00000ec0
0x00000131
0x00000131
0xffffffff
007d000c
007d0ec8
007d0ecc
007d0ed0
007d0ff8
007d1000
ec0
130
> ec0
= ff0-130
00000131
00000000
00000000
0042ee08
00000081
00000100
00000002
00000001
4 個 0xfd
fill 0xcd
4 個 0xfd
00000131
"ioinit.c"
user_ptr
nsize
(100h)
/* Memory block identification */
#define _FREE_BLOCK 0
#define _NORMAL_BLOCK 1
#define _CRT_BLOCK 2
#define _IGNORE_BLOCK 3
#define _CLIENT_BLOCK 4
#define _MAX_BLOCKS 5
0x100 + 0x24 + 4*2
➔ 0x12C
➔ 0x130

Boolan 博览

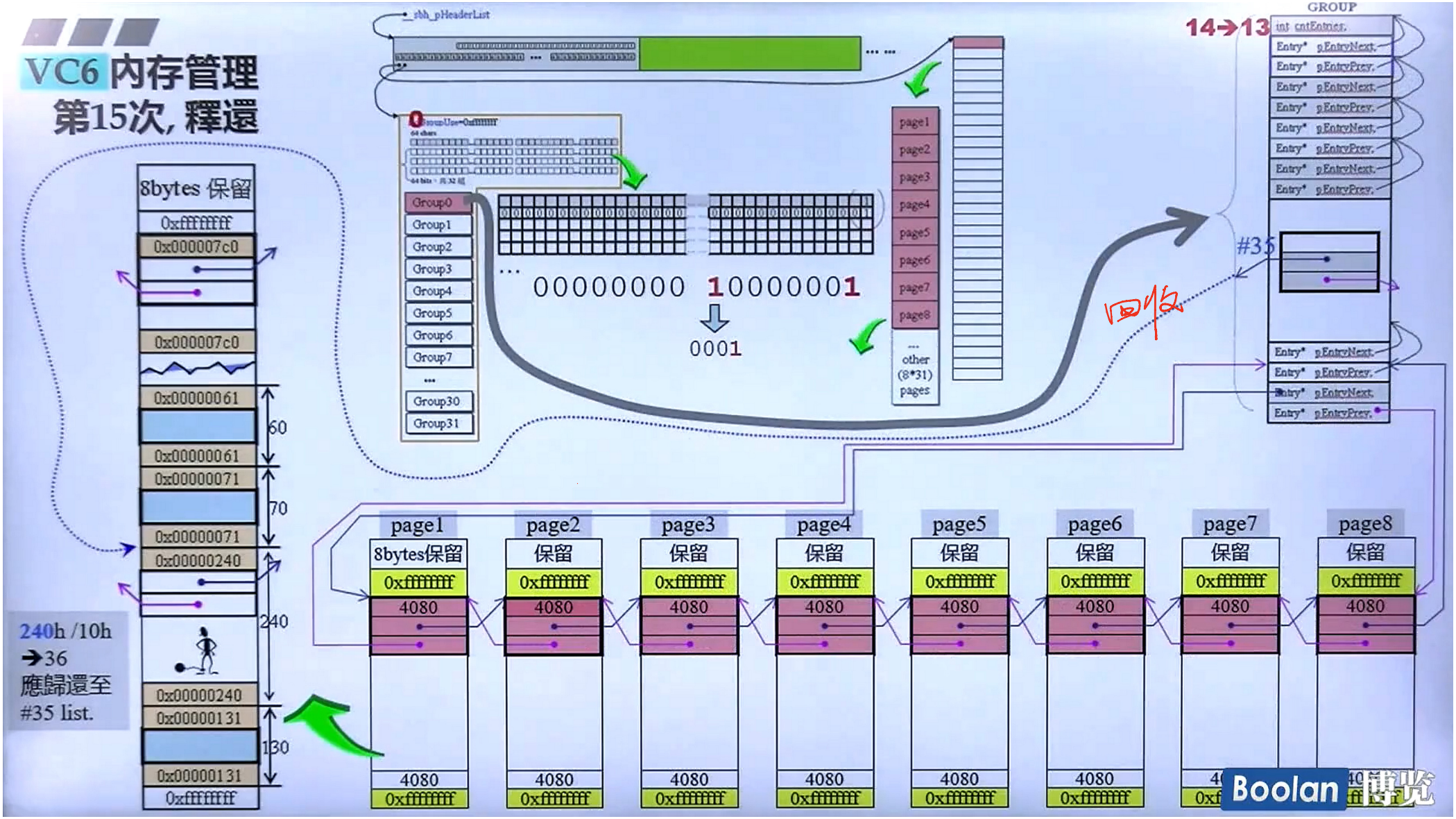

VC6 內存管理
首次分配
由 ioinit.c, line#81申請
100h, 區塊大小130h,
理應由 #18 lists 供應
HeapAlloc(,,16*sizeof(HEADER))
p=VirtualAlloc(0 ,1Mb,
MEM_RESERVE, ...)
VirtualAlloc(addr ,32Kb,
MEM_COMMIT,...)
HeapAlloc(_crtheap,,sizeof(REGION));
GROUP
Group0
Group1
Group2
Group3
Group4
Group5
Group6
Group7
Group30
Group31
page1
page2
page3
page4
page5
page6
page7
page8
other (8*31) pages
00000131
"ioinit.c"
00000081
00000100
00000002
00000001
fdfdfdfd
fill with 0xcd
100
fdfdfdfd
00000131
8bytes保留
0xffffffff
0x00000ec0
ec0
0x00000ec0
0x00000131
130
0x00000131
0xffffffff
保留
4080
Boolan 博览

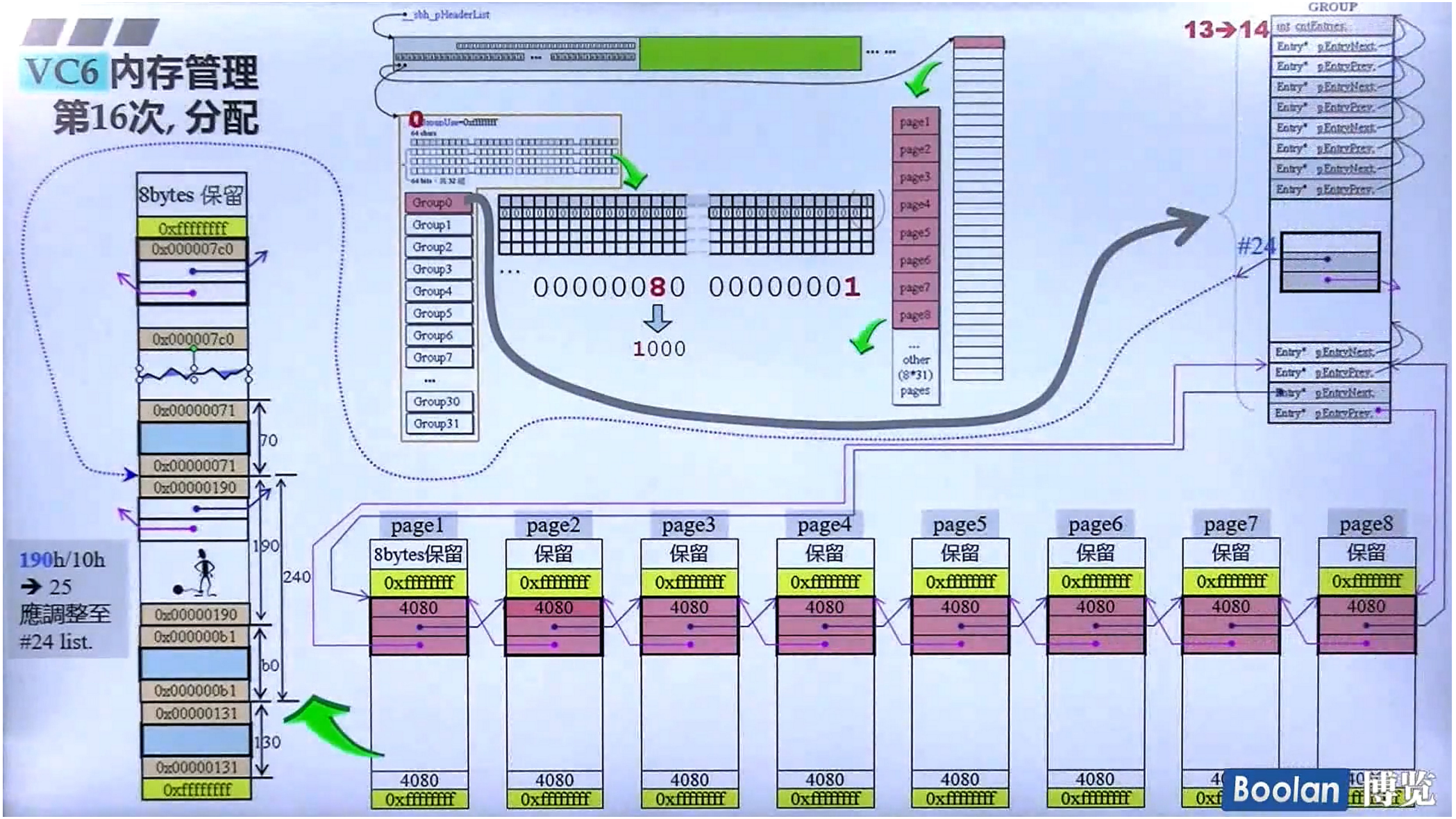

VC6 内存管理
第2次, 分配
HeapAlloc(,,16*sizeof(HEADER))
p=VirtualAlloc(0 ,1Mb,
MEM_RESERVE, ...)
VirtualAlloc(addr ,32Kb,
MEM_COMMIT,...)
c80
=ec0-240
8bytes 保留
0xffffffff
0x00000c80
0x00000c80
0x00000241
0x00000241
0x00000131
0x00000131
0xffffffff
c80
240
130
Group0
Group1
Group2
Group3
Group4
Group5
Group6
Group7
Group30
Group31
page1
page2
page3
page4
page5
page6
page7
page8
other (8*31) pages
GROUP
Entry* pEntryNext;
Entry* pEntryPrev;
8bytes保留
保留
4080
0xffffffff
Boolan 博览

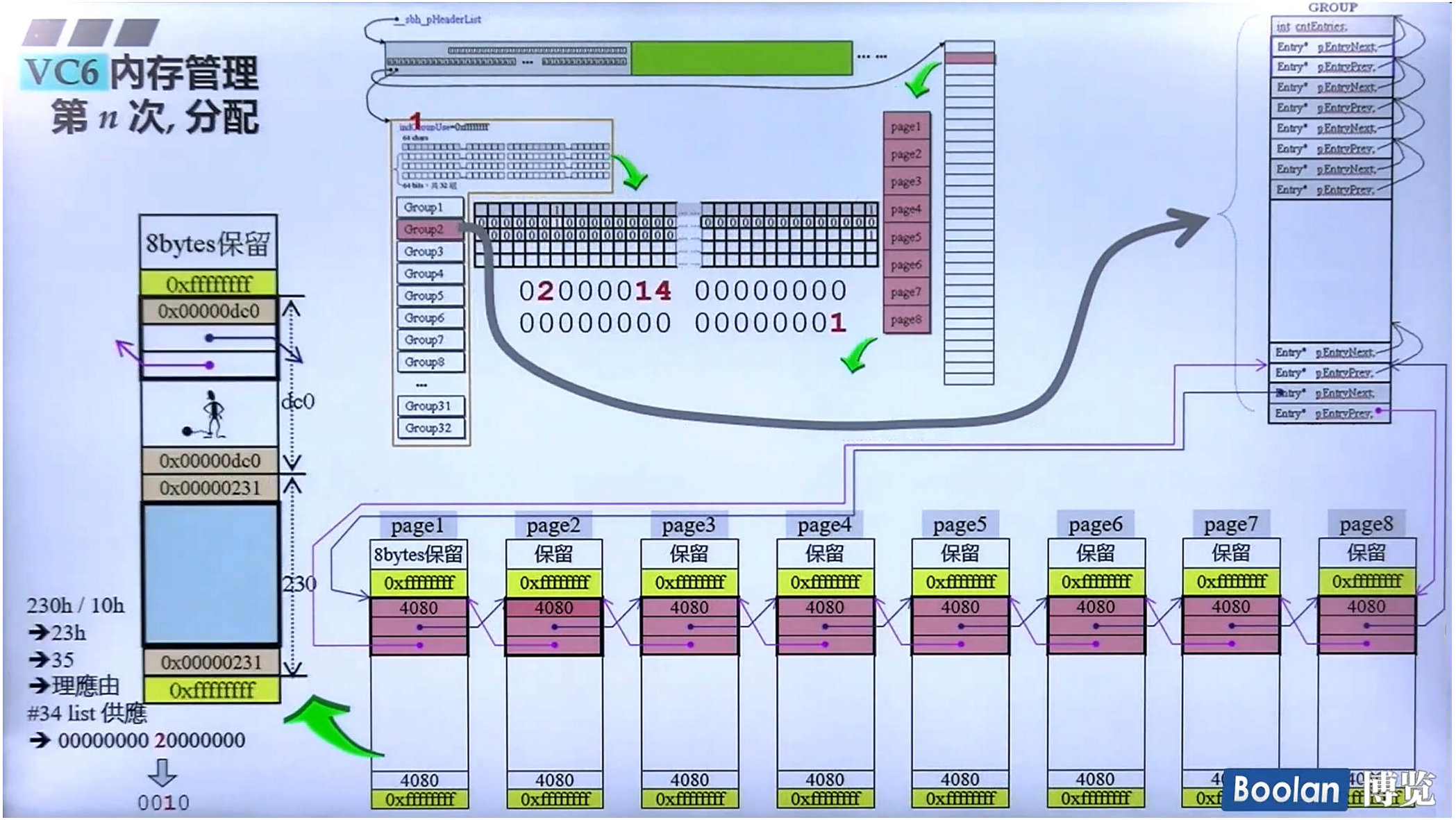
VC6 内存管理
第3次, 分配



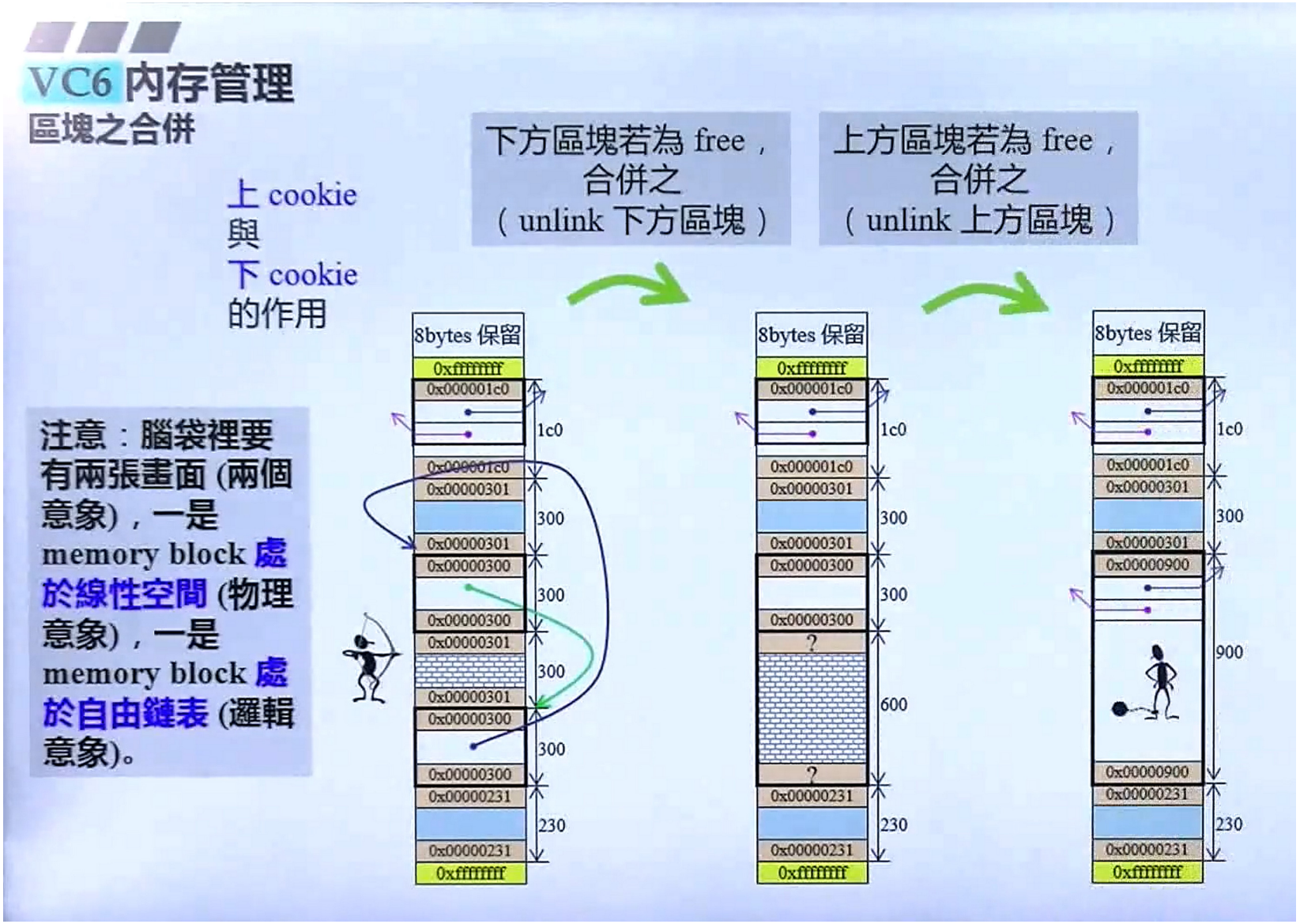
VC6 內存管理
第15次, 釋還




VC6 內存管理
第16次, 分配
8bytes 保留
0xffffffff
0x000007c0
0x000007c0
0x00000071
0x00000071
0x00000190
0x00000190
0x000000b1
0x000000b1
0x00000131
0x00000131
0xffffffff
70
190
240
b0
130
190h/10h
➔ 25
應調整至
#24 list.
_sbh_pHeaderList
Group0
Group1
Group2
Group3
Group4
Group5
Group6
Group7
Group30
Group31
00000080 00000001
1000
page1
page2
page3
page4
page5
page6
page7
page8
other (8*31) pages
GROUP
13→14
#24
page1
page2
page3
page4
page5
page6
page7
page8
8bytes保留
保留
0xffffffff
4080
Boolan 博览


VC6 内存管理
第 n 次, 分配
8bytes保留
0xffffffff
0x00000dc0
dc0
0x00000dc0
0x00000231
230
0x00000231
0xffffffff
230h / 10h
➔23h
➔35
➔理應由
#34 list 供應
➔ 00000000 20000000
0010
Group1
Group2
Group3
Group4
Group5
Group6
Group7
Group8
Group31
Group32
02000014 00000000
00000000 00000001
page1
page2
page3
page4
page5
page6
page7
page8
GROUP
int cntEntries;
Entry* pEntryNext;
Entry* pEntryPrev;
8bytes保留
保留
0xffffffff
4080
Boolan 博览

8. 内存分配精解4



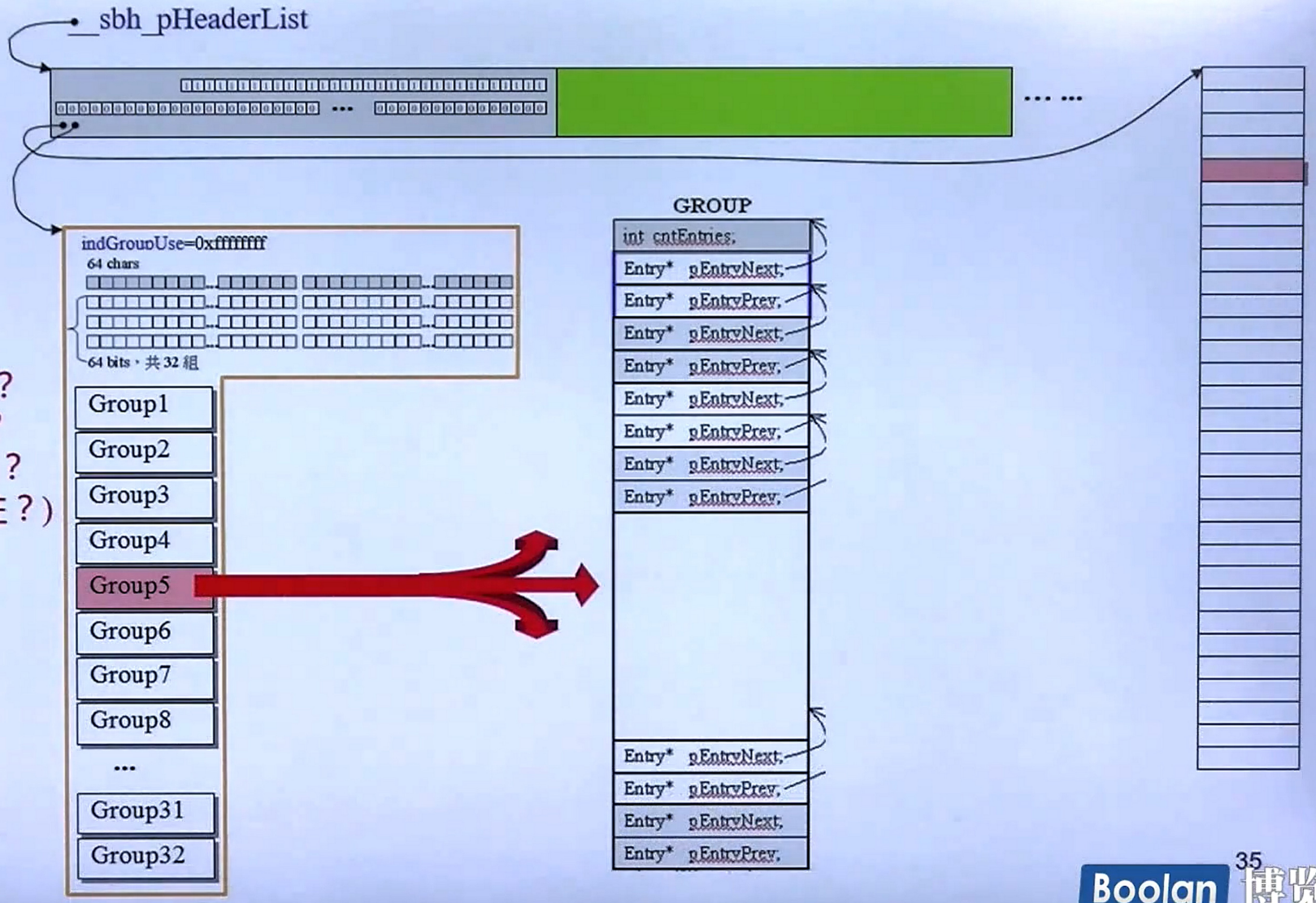
通过上 cookie 可以得到下一个区块的上 cookie，进而通过该上 cookie 的最后一个字节判断该区块是否被使用；
user_ptr + 4 得到当前区间的上 cookie，也就得到了当前区块的长度 len，再往下移动 len 即可得到下一个区间的上 cookie
同理，通过下 cookie，我们可以得到上一个区间的头 cookie，进而通过该上 cookie 的最后一个字节判断该区块是否被使用；
user_ptr + 8 可以得到上一个区块的下 cookie，也就得到了上一个区间的长度 len，再往上移动 len 即可得到上一个区块的上 cookie

VC6 內存管理

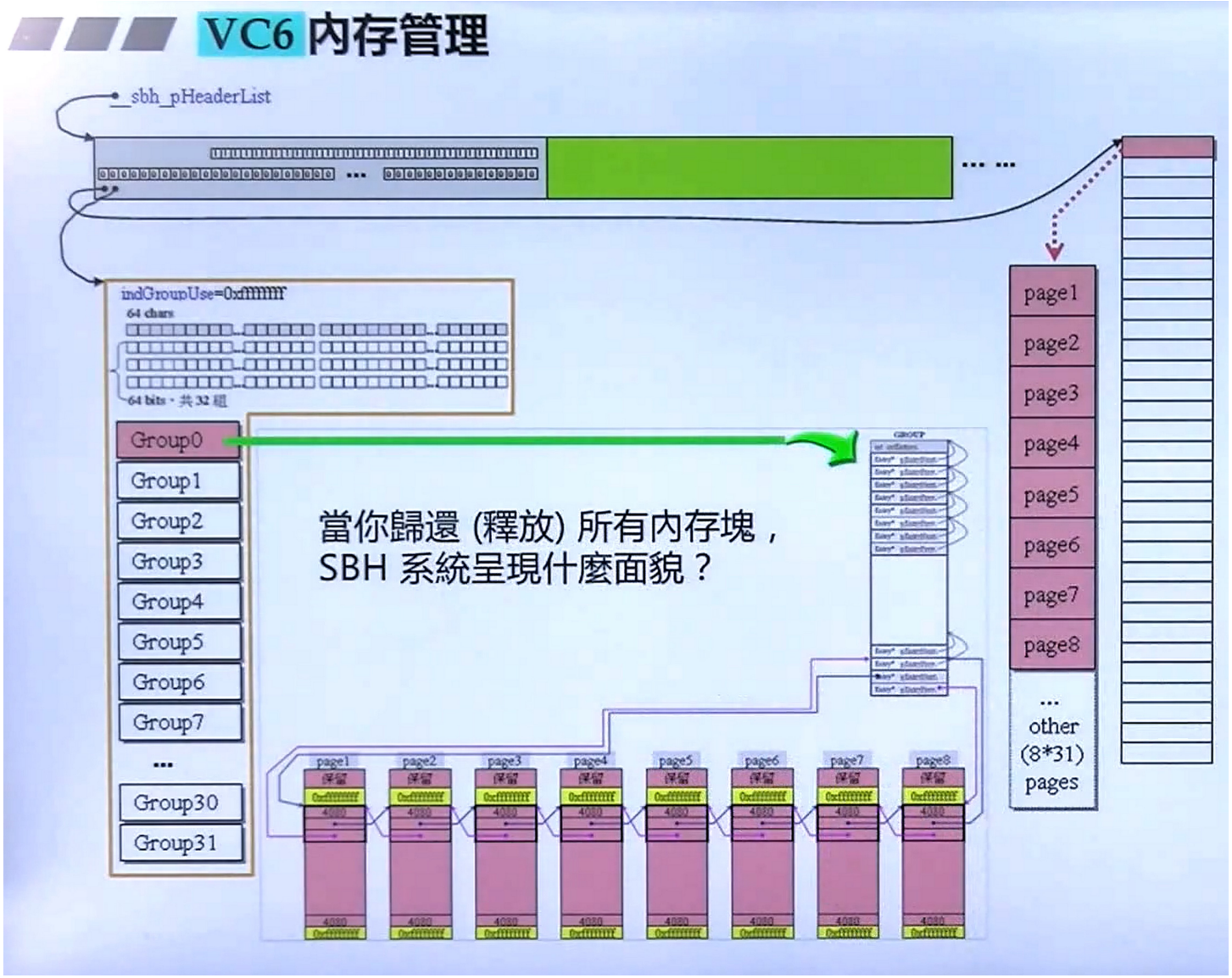
free(p)

p 花落誰家？

落在哪個 Header 內？
落在哪個 Group 內？
落在哪個 free-list 內？
(被哪個 free-list 鏈住？)



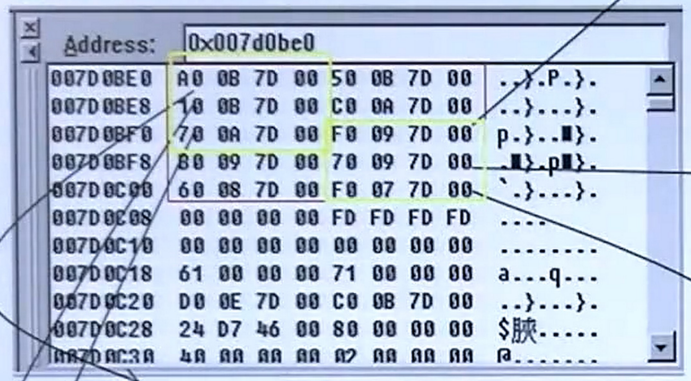
Boolan 博览



9. Main()生前所有内存分配之实例观察与解释

_environ 是 pointer to pointer table, table 中的每個 entry 都是 pointer to string which represents an environment variable.

_environ : 0x007d0be0



0

```
007D0BA0  54 4D 50 3D 43 3A 5C 57  TMP=C:\W
007D0BA8  49 4E 44 4F 57 53 5C 54  INDOWS\T
007D0BB0  45 4D 50 00 FD FD FD FD  EMP.
```

長度 20

1

```
007D0B50  54 45 4D 50 3D 43 3A 5C  TEMP=C:\
007D0B58  57 49 4E 44 4F 57 53 5C  WINDOWS\
007D0B60  54 45 4D 50 00 FD FD FD  TEMP
```

長度21

2

```
007D0B10  50 52 4F 4D 50 54 3D 24  PROMPT=$
007D0B18  70 24 67 00 FD FD FD FD  p$g.
```

長度12

3

```
007D0AC0  77 69 6E 62 6F 6F 74 64  winbootd
007D0AC8  69 72 3D 43 3A 5C 57 49  ir=C:\WI
007D0AD0  4E 44 4F 57 53 00 FD FD  NDOWS.
007D0AD8  FD FD 00 00 00 00 00 00  ....
```

長度22

4

```
007D0A70  43 4F 4D 53 50 45 43 3D  COMSPEC=
007D0A78  43 3A 5C 57 49 4E 44 4F  C:\WINDO
007D0A80  57 53 5C 43 4F 4D 4D 41  WS\COMMA
007D0A88  4E 44 2E 43 4F 4D 00 FD  ND.COM.
007D0A90  FD FD FD 00 00 00 00 00  .....
```

長度31

5

```
007D09F0  50 41 54 48 3D 43 3A 5C  PATH=C:\
007D09F8  57 49 4E 44 4F 57 53 3B  WINDOWS;
007D0A00  43 3A 5C 57 49 4E 44 4F  C:\WINDO
007D0A08  57 53 5C 43 4F 4D 4D 41  WS\COMMA
007D0A10  4E 44 3B 45 3A 5C 55 54  ND;E:\UT
007D0A18  49 4C 49 54 59 3B 43 3A  ILITY;C:
007D0A20  5C 50 52 4F 47 52 41 7E  \PROGRA~
007D0A28  31 5C 43 4F 4D 4D 4F 4E  1\COMMON
007D0A30  7E 31 5C 4D 55 56 45 45  ~1\MUVEE
007D0A38  54 7E 31 5C 30 33 30 36  T~1\0306
007D0A40  32 35 00 FD FD FD FD 00  25.
```

長度83

6

```
007D09B0  43 4D 44 4C 49 4E 45 3D  CMDLINE=
007D09B8  57 49 4E 00 FD FD FD FD  WIN.
```

長度12

7

```
007D0970  77 69 6E 64 69 72 3D 43  windir=C
007D0978  3A 5C 57 49 4E 44 4F 57  :\WINDOW
007D0980  53 00 FD FD FD FD 00 00  S.
```

長度18

8

```
007D0860  5F 41 43 50 5F 50 41 54  _ACP_PAT
007D0868  48 3D 43 3A 5C 4D 53 44  H=C:\MSD
007D0870  45 56 5C 43 4F 4D 4D 4F  EV\COMMO
007D0878  4E 5C 4D 53 44 45 56 39  N\MSDEV9
007D0880  38 5C 42 49 4E 3B 43 3A  8\BIN;C:
007D0888  5C 6D 73 64 65 76 5C 56  \msdev\V
007D0890  43 39 38 5C 42 49 4E 3B  C98\BIN;
007D0898  43 3A 5C 4D 53 44 45 56  C:\MSDEV
007D08A0  5C 43 4F 4D 4D 4F 4E 5C  \COMMON\
007D08A8  54 4F 4F 4C 53 3B 43 3A  TOOLS;C:
```

長度??

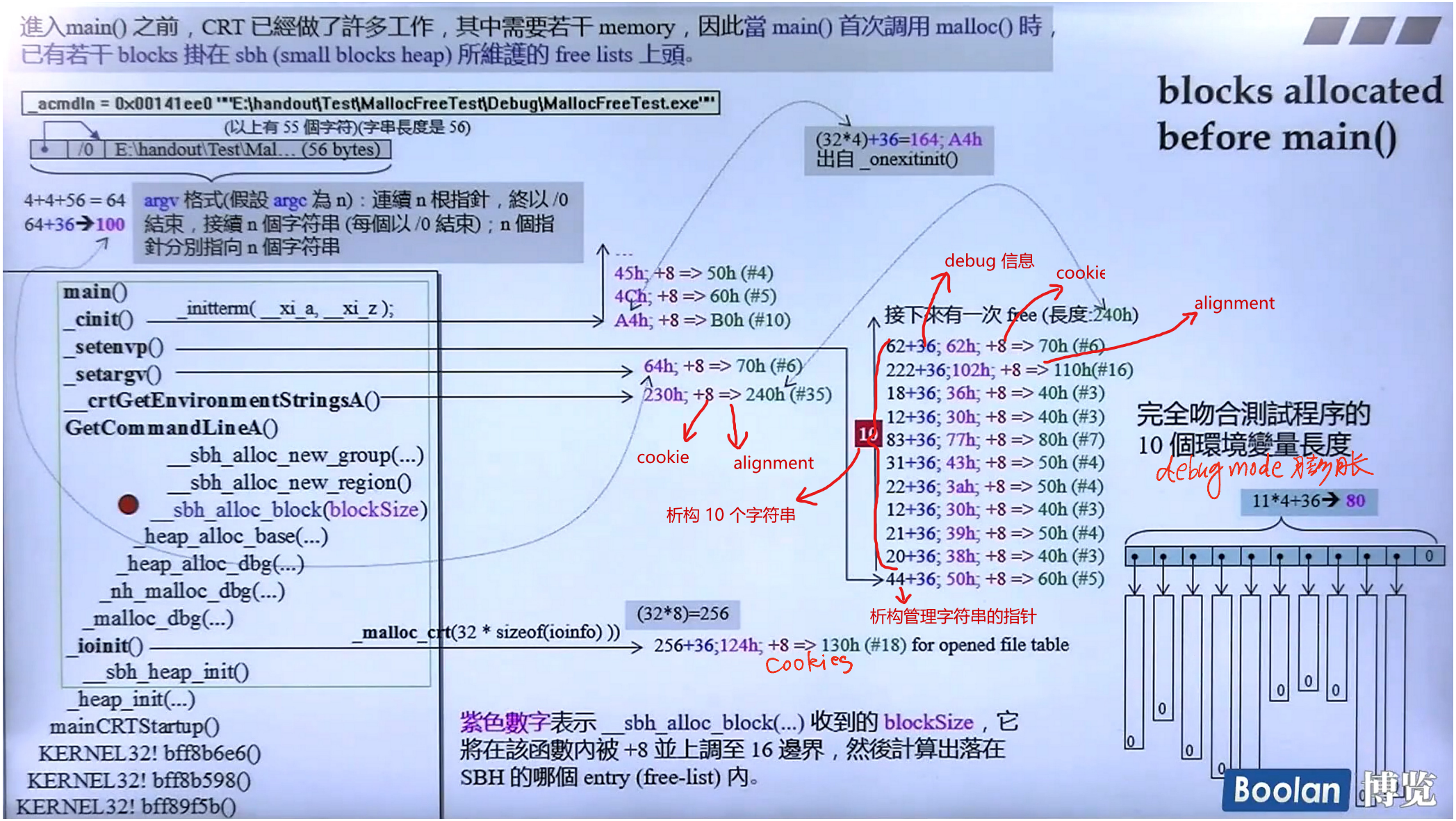
9

```
007D07F0  5F 41 43 50 5F 4C 49 42  _ACP_LIB
007D07F8  3D 43 3A 5C 6D 73 64 65  =C:\msde
007D0800  76 5C 56 43 39 38 5C 4C  v\VC98\L
007D0808  49 42 3B 43 3A 5C 6D 73  IB;C:\ms
007D0810  64 65 76 5C 56 43 39 38  dev\VC98
007D0818  5C 4D 46 43 5C 4C 49 42  \MFC\LIB
007D0820  3B 63 3A 5C 62 6F 6F 73  ;c:\boos
007D0828  74 5C 6C 69 62 00 FD FD  t\lib.
007D0830  FD FD 00 00 00 00 00 00  ......
```

長度62

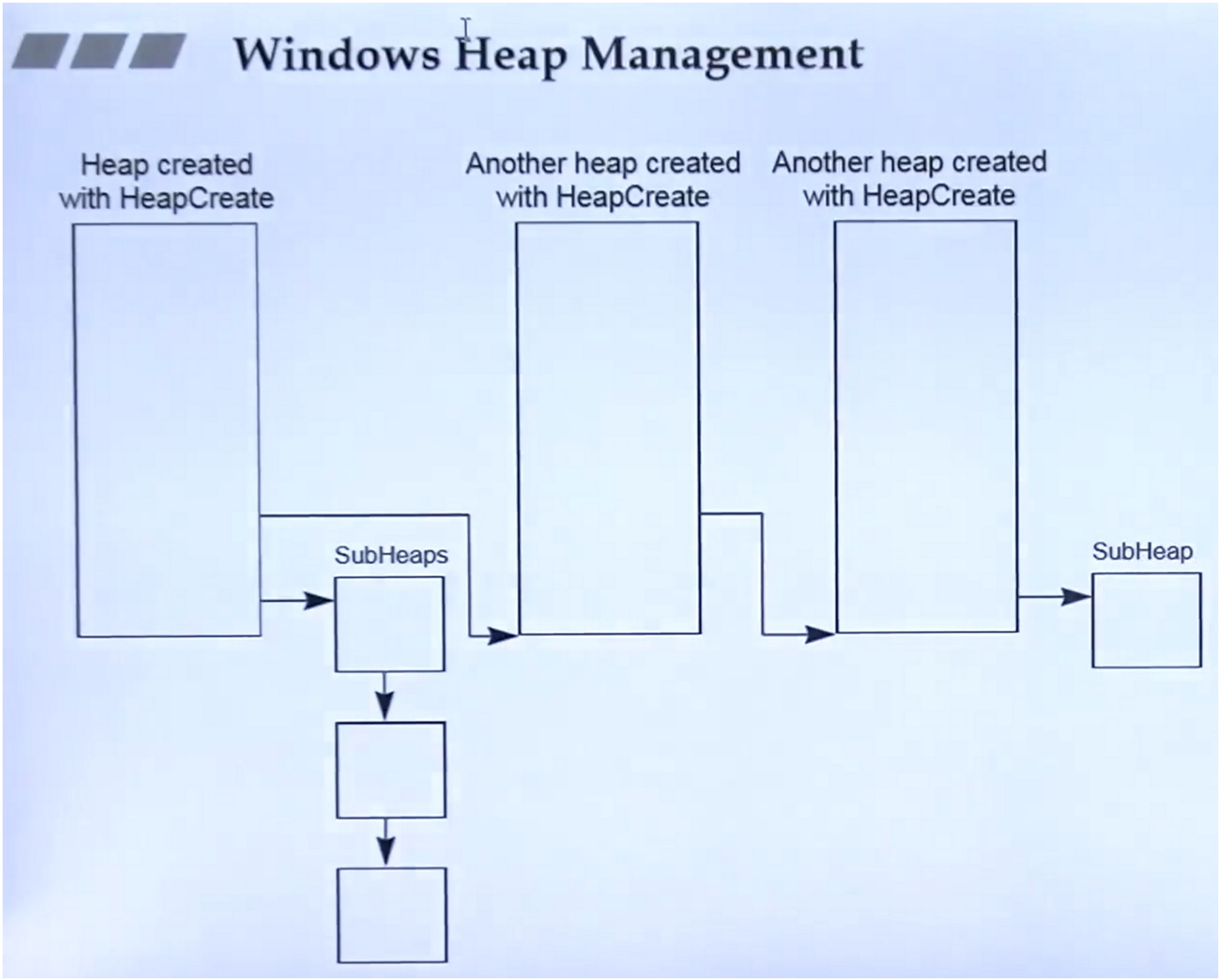
注意长度包含最后的 0x00

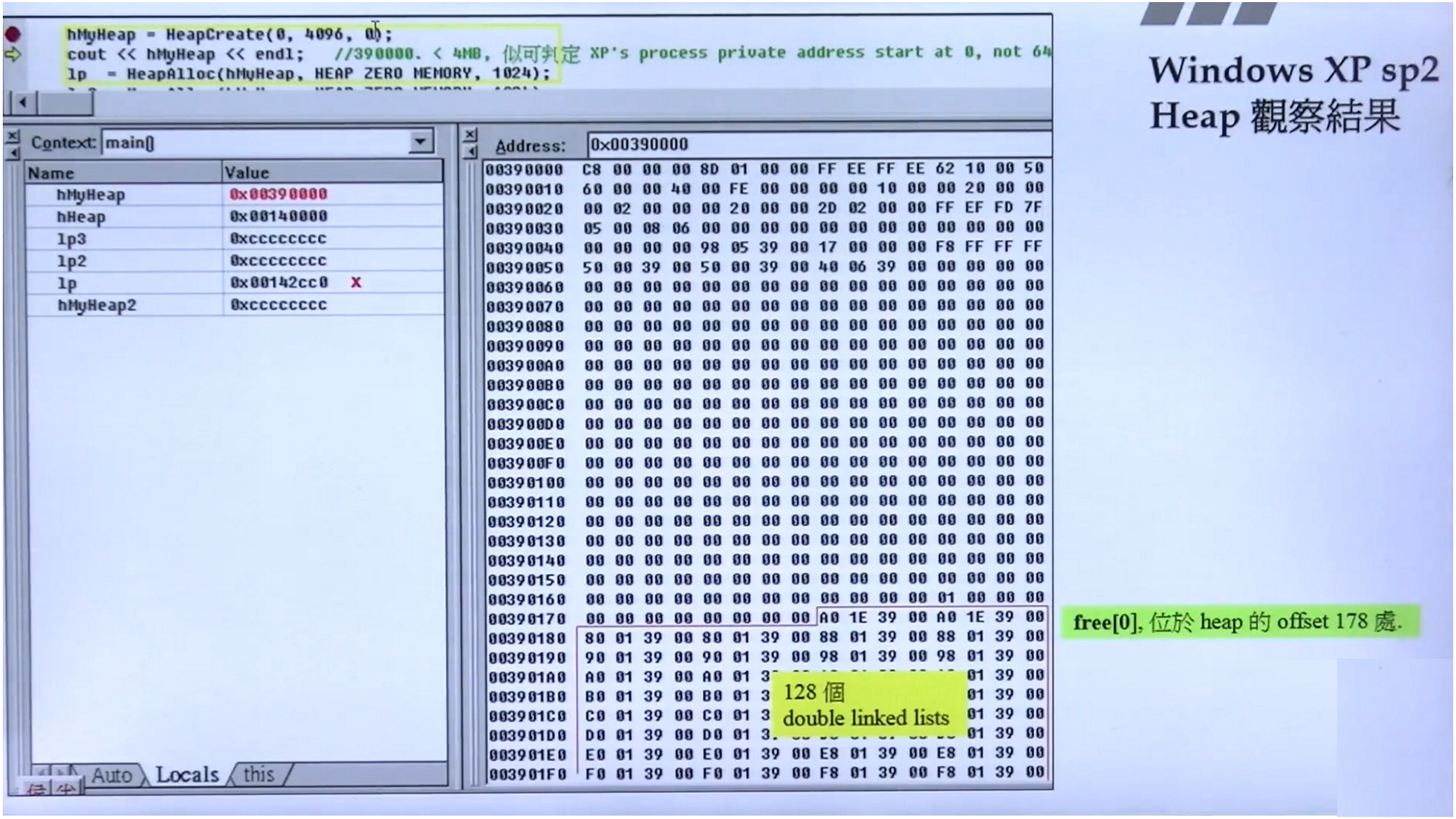
blocks allocated before main()



10. HeapAlloc()角色与影响

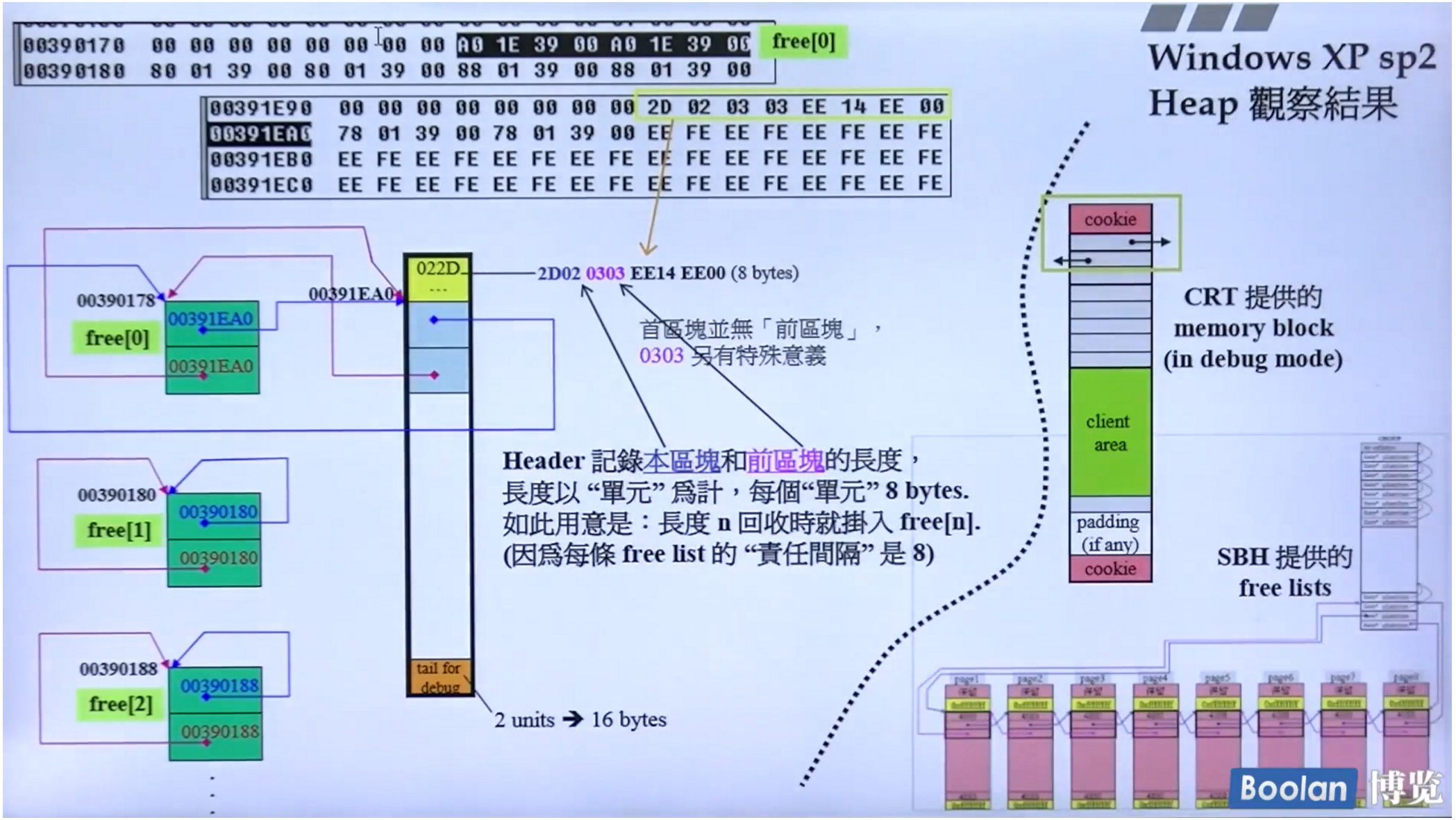
Windows Heap Management



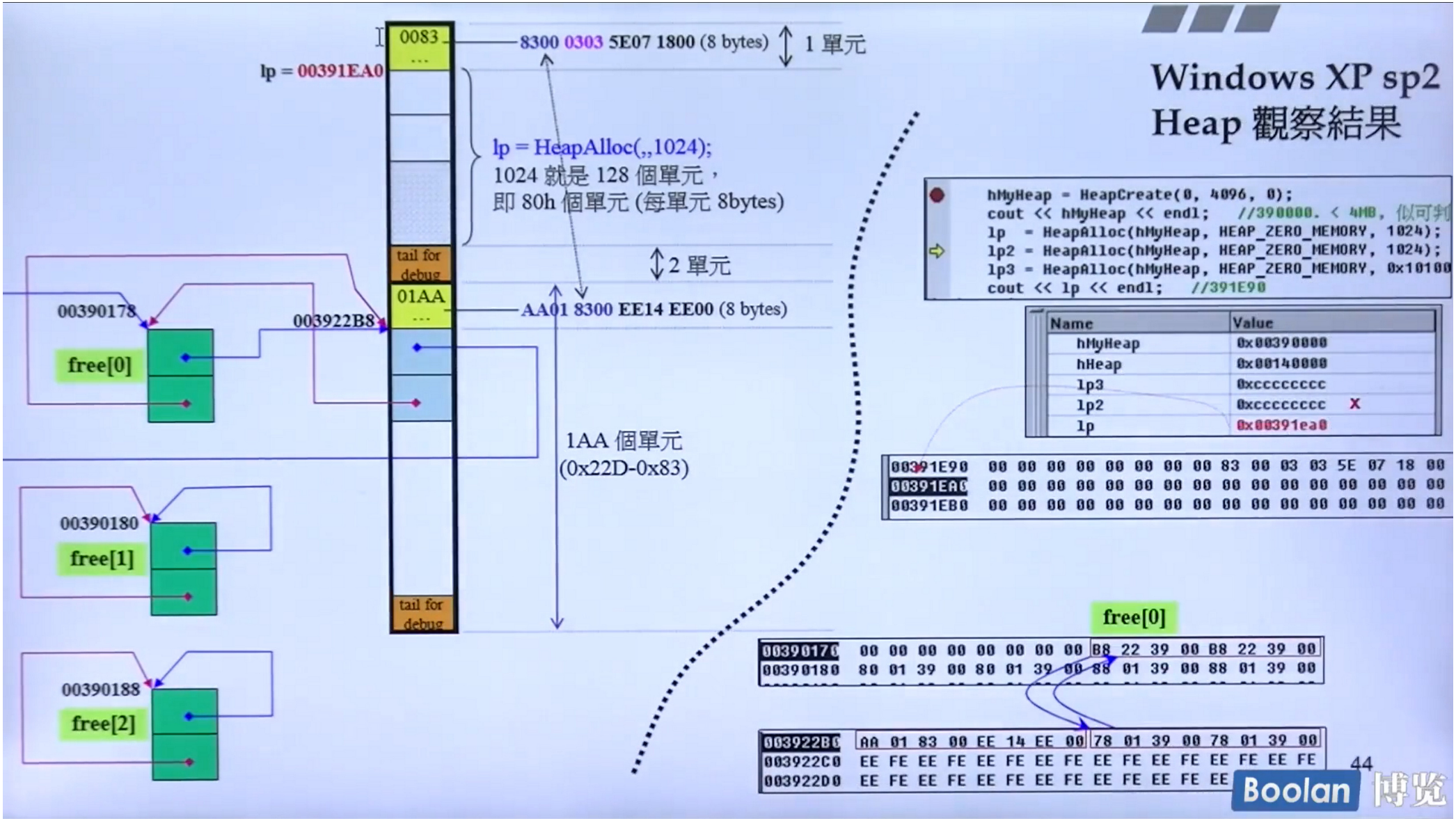

hMyHeap = HeapCreate(0, 4096, 0);
cout << hMyHeap << endl; //390000. < 4MB, 似可判定 XP's process private address start at 0, not 64
lp = HeapAlloc(hMyHeap, HEAP ZERO MEMORY, 1024);
Context: main()
Name
Value
hMyHeap
0x00390000
hHeap
0x00140000
lp3
0xcccccccc
lp2
0xcccccccc
lp
0x00142cc0
hMyHeap2
0xcccccccc
Auto
Locals
this
Address: 0x00390000
00390000 C8 00 00 00 8D 01 00 00 FF EE FF EE 62 10 00 50
00390010 60 00 00 40 00 FE 00 00 00 00 10 00 00 20 00 00
00390020 00 02 00 00 00 20 00 00 2D 02 00 00 FF EF FD 7F
00390030 05 00 08 06 00 00 00 00 00 00 00 00 00 00 00 00
00390040 00 00 00 00 98 05 39 00 17 00 00 00 F8 FF FF FF
00390050 50 00 39 00 50 00 39 00 40 06 39 00 00 00 00 00
00390060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00390070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00390080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00390090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
003900A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
003900B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
003900C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
003900D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
003900E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
003900F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00390100 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00390110 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00390120 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00390130 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00390140 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00390150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00390160 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00 00
00390170 00 00 00 00 00 00 00 00 A0 1E 39 00 A0 1E 39 00
00390180 80 01 39 00 80 01 39 00 88 01 39 00 88 01 39 00
00390190 90 01 39 00 90 01 39 00 98 01 39 00 98 01 39 00
003901A0 A0 01 39 00 A0 01 3
003901B0 B0 01 39 00 B0 01 3
003901C0 C0 01 39 00 C0 01 3
003901D0 D0 01 39 00 D0 01 3
003901E0 E0 01 39 00 E0 01 39 00 E8 01 39 00 E8 01 39 00
003901F0 F0 01 39 00 F0 01 39 00 F8 01 39 00 F8 01 39 00
128 個
double linked lists
Windows XP sp2
Heap 觀察結果
free[0], 位於 heap 的 offset 178 處.

free[0], 位於 heap 的 offset 178 處.
Windows XP sp2
Heap 觀察結果
size: 022D
free[127]
128 個 free lists，
也就是 128 對指針，
0x390178 + 128 * 8
= 0x390578
42
Boolan 博

Windows XP sp2 Heap 觀察結果



Boolan 博览



11. Io_init() - startup 的第二项大工程

_io_init()

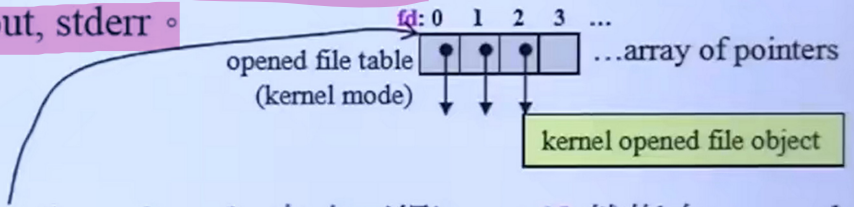
I/O 泛指程序和外界的各種交互作用，包括 file, pipe, network, console, semaphore 等等，或泛指能被 OS 理解爲 file 的任何事務 — file 在此是一個廣義概念。

```
FILE* fp = fopen("xxx.dat", "wb");
```

```
struct _iobuf { … };
typedef struct _iobuf FILE;
```

C 通過一個 pointer to **FILE** 來進行 file 操作。在 OS 層面，Linux 對應於 **FILE** 的是 File Descriptor (**fd**)，Windows 對應的則是 **file handle**. 二者都用來映射 kernel file object。

fd 具體是個 index of **opened file table** —process 個別擁有的這個 table 是個 array of pointers，每個 pointer 指向一個 kernel opened object。當 client 開啓一個 file，OS 會建立一個 (kernel) **opened file object** 並找到上述 table 中的一個 idle entry 指向之，然後以該 entry 的 index 做爲 **fd**。此 table 位於 kernel mode，因此 client 即使擁有 **fd** 亦無法獲得 table address. Linux 的 **fd** 0,1,2 分別代表 stdin, stdout, stderr。



C 的 **FILE** 與 Linux 的 **fd** 必有一對一關係。只要有 table address p (kernel mode 中才可得)，p+**fd** 就指向 opened file table 的某個 entry，從而可得 kernel file object。

Windows 的 **file handle** 和 Linux 的 **fd** 大同小異，但 **handle** 不是 index，而是 index 經某種變換後的結果。

I/O initialization 就是要在 **client space** 中建立起 stdin, stdout, stderr 及其對應的 **FILE**s，使程式進入 main() 之後立即可用 printf(), scanf() 等函式。

Boolan 博览

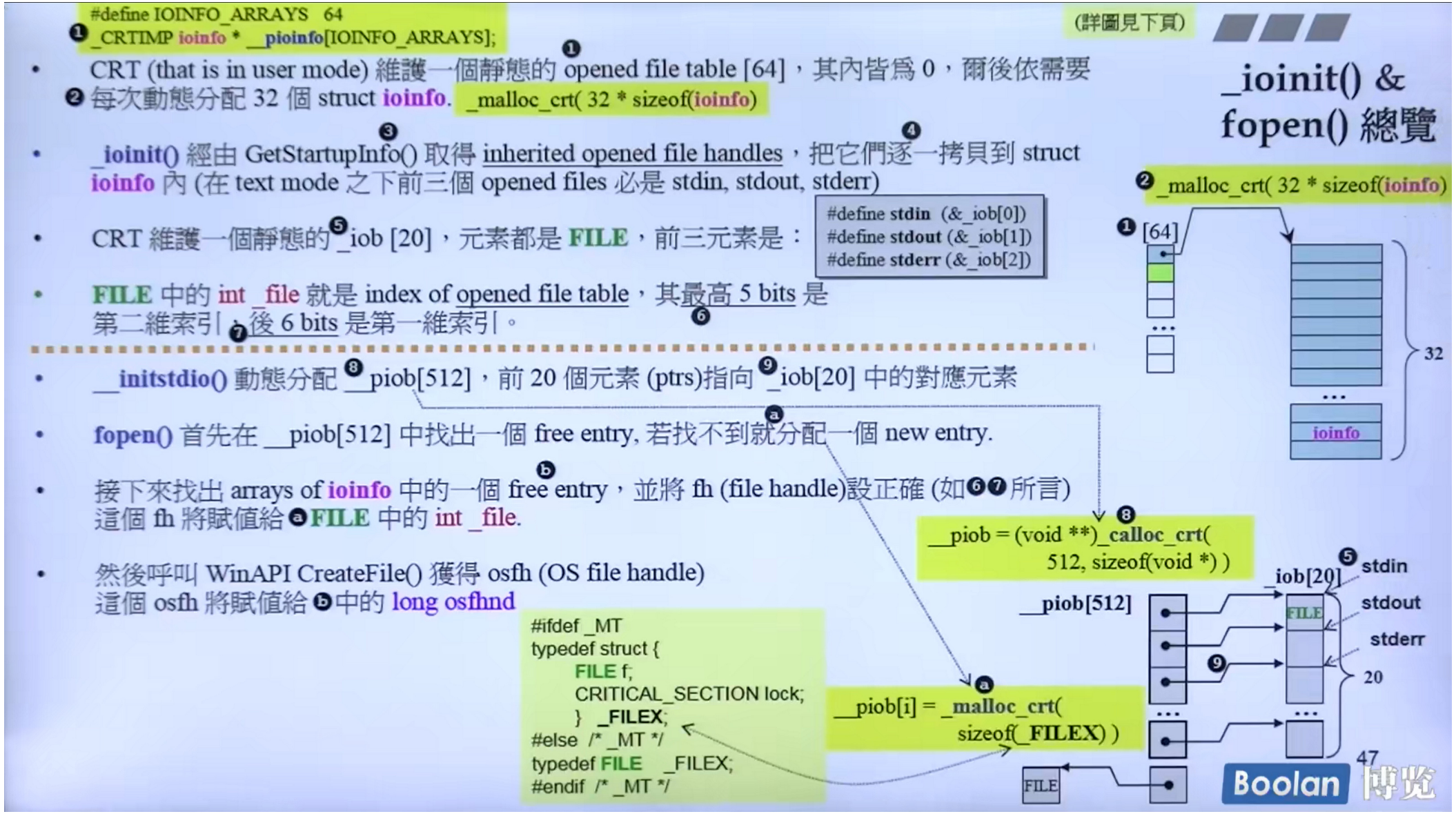
_io_init()

```
    ExitProcess(code)
    _initterm(,,)   //do terminators
      __endstdio(void)
    _initterm(,,)   //do pre-terminators
  doexit(code, 0, 0)
exit(code)
main()
  _initterm(,,) //do C++ initializations
    __initstdio(void)
  _initterm(,,) //do initializations
_cinit()
_setenvp()
_setargv()
__crtGetEnvironmentStringsA()
GetCommandLineA()
            __sbh_alloc_new_group(...)
            __sbh_alloc_new_region()
          __sbh_alloc_block(...)
        _heap_alloc_base(...) //size:292
      _heap_alloc_dbg(...)
    _nh_malloc_dbg(...)
  _malloc_dbg(...) //size:256
_ioinit()
  __sbh_heap_init()
_heap_init(...)
mainCRTStartup()
```

ioinit.c

```
/***
*Purpose:
*       Allocates and initializes initial array(s) of ioinfo structs. Then,
*       obtains and processes information on inherited file handles from the
*       parent process (e.g., cmd.exe).
*
*       Obtains the StartupInfo structure from the OS. The inherited file
*       handle information is pointed to by the lpReserved2 field. The format
*       of the information is as follows:
*           bytes 0 thru 3 - integer value, say N, which is the
*                            number of handles information is passed about
*           bytes 4 thru N+3   - the N values for osfile
*           bytes N+4 thru 5*N+3  - N double-words, the N OS HANDLE values
*                            being passed
*       Next, osfhnd and osfile for the first three ioinfo structs,
*       corrsponding to handles 0, 1 and 2, are initialized as follows:
*           If the value in osfhnd is INVALID_HANDLE_VALUE, then try to
*           obtain a HANDLE by calling GetStdHandle, and call GetFileType to
*           help set osfile. Otherwise, assume _osfhndl and _osfile are
*           valid, but force it to text mode (standard input/output/error
*           are to always start out in text mode).
*       Notes:
*           1. In general, not all of the passed info from the parent process
*              will describe open handles! If, for example, only C handle 1
*              (STDOUT) and C handle 6 are open in the parent, info for C
*              handles 0 thru 6 is passed to the the child.
*           2. Care is taken not to 'overflow' the arrays of ioinfo structs.
*           3. See exec\dospawn.c for the encoding of the file handle info
*              to be passed to a child process.
*Entry:
*       No parameters: reads the STARTUPINFO structure.
*Exit:
*       No return value.
*Exceptions:
*******************************************************************************/
```

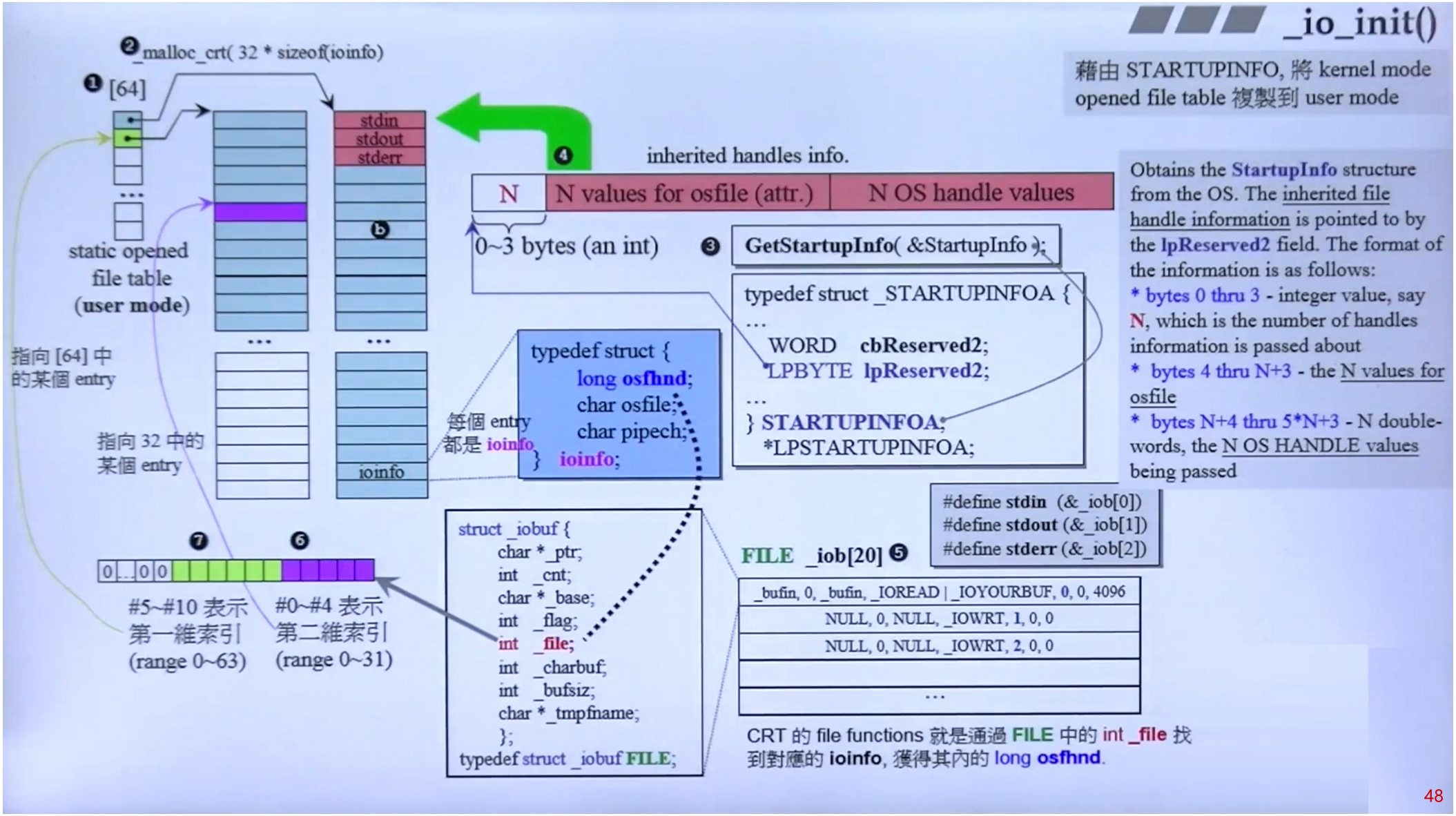

Boolan



12. C_init() startup的第三大工程

_io_init()

藉由 STARTUPINFO, 將 kernel mode opened file table 複製到 user mode

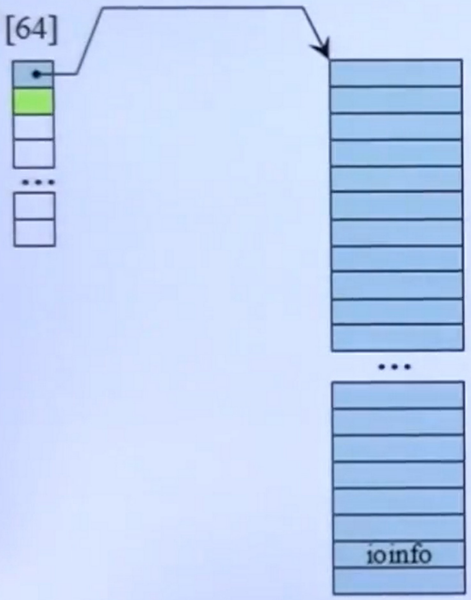


_io_init()

```
#define IOINFO_ARRAYS   64
_CRTIMP ioinfo * __pioinfo[IOINFO_ARRAYS];
```

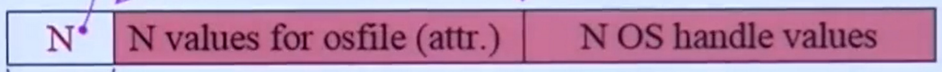
❷ _malloc_crt(32 * sizeof(ioinfo)

❶ [64]




```
#0080    /* * Process inherited file handle information, if any  */
#0081    GetStartupInfo( &StartupInfo );
#0082    if ( (StartupInfo.cbReserved2 != 0) &&
#0083         (StartupInfo.lpReserved2 != NULL) )
#0084    {
#0085        /* * Get the number of handles inherited. */
#0086        cfi_len = *(UNALIGNED int *)(StartupInfo.lpReserved2);

#0087        /* * Set pointers to the start of the passed file info and OS
#0088         * HANDLE values.  */
#0089        posfile = (char *)(StartupInfo.lpReserved2) + sizeof( int );
#0090        posfhnd = (UNALIGNED long *)(posfile + cfi_len);
```



0~3 bytes (an int)

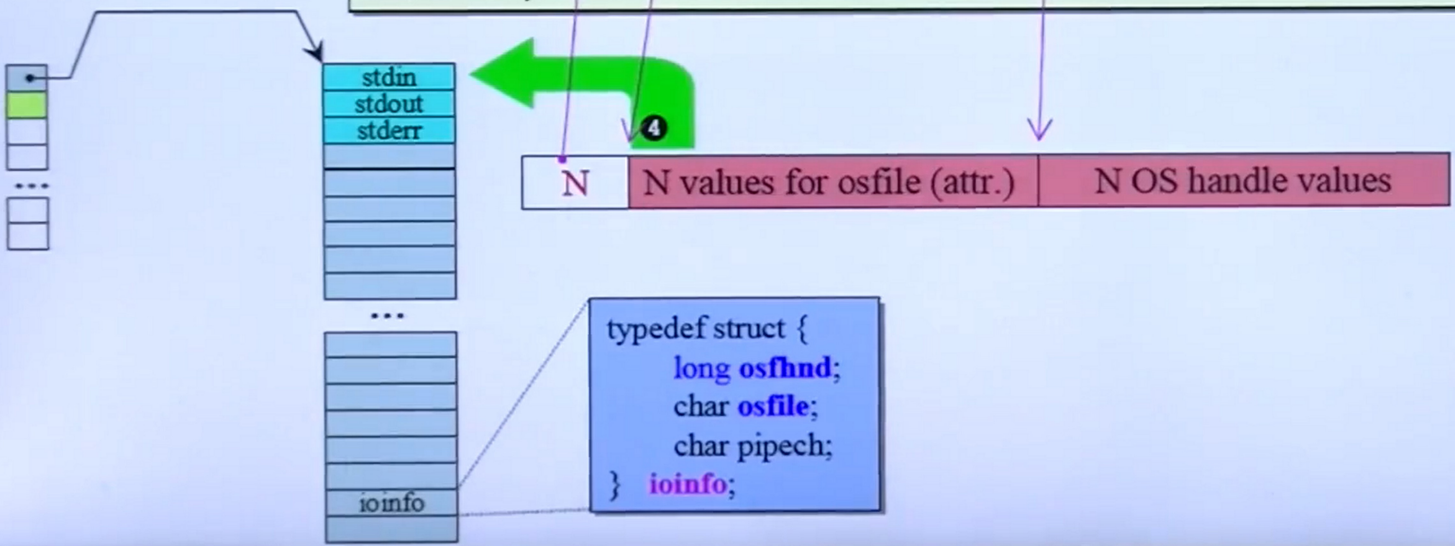
❸ GetStartupInfo(&StartupInfo);

```
typedef struct _STARTUPINFOA {
…
    WORD    cbReserved2;
    LPBYTE  lpReserved2;
…
} STARTUPINFOA;
  *LPSTARTUPINFOA;
```


Boolan 博览

_io_init()

```
#0119       for ( fh = 0 ; fh < cfi_len ; fh++, posfile++, posfhnd++ ) {
#0120         /*
                ...
#0127          */
#0128         if ( (*posfhnd != (long)INVALID_HANDLE_VALUE) &&
#0129             (*posfile & FOPEN) &&
#0130             ((*posfile & FPIPE) ||
#0131              (GetFileType( (HANDLE)*posfhnd ) != FILE_TYPE_UNKNOWN)) )
#0132         {
#0133           pio = _pioinfo( fh );
#0134           pio->osfhnd = *posfhnd;
#0135           pio->osfile = *posfile;
#0136         }
#0137       }
#0138     }
```



Boolan 博览

file test

```
#include <stdio.h>
#include <iostream>
using namespace std;

int main()
{
  cout << stdin << endl;          //00414538
  cout << stdin->_file << endl;   //0
  cout << stdout << endl;         //00414558
  cout << stdout->_file << endl;  //1
  cout << stderr << endl;         //00414578
  cout << stderr->_file << endl;  //2

  FILE* fp = fopen("xxx.dat", "wb");
  if (fp == NULL) return -1;
  cout << fp << endl;             //00414598
  cout << fp->_file << endl;      //3

  fwrite("abcde", 5, 1, fp);
  fclose(fp);
  cout << fp << endl;             //00414598
  cout << fp->_file << endl;      //3
```

fclose() 會歸還 file handle!

```
#define stdin  (&_iob[0])
#define stdout (&_iob[1])
#define stderr (&_iob[2])
```

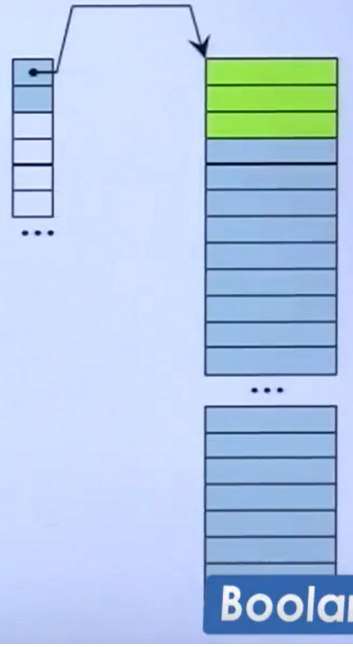
sizeof(FILE): 0x20
所以 stdin, stdout, stderr (都是指針) 相差 0x20

```
  fp = fopen("yyy.dat", "wb");
  if (fp == NULL) return -1;
  cout << fp << endl;             //00414598
  cout << fp->_file << endl;      //3
```

```
  fwrite("fghij", 5, 1, fp);
  fclose(fp);
  cout << fp << endl;             //00414598
  cout << fp->_file << endl;      //3

  int i;
  FILE* f[60];
  char buffer[20];
  for (i=0; i< 60; ++i) {
     f[i] = fopen(_itoa(i,buffer,10), "wb");
     cout << f[i]->_file << endl;      //3 4 5 6 ... 62
  }
  return 0;
}
```

本例先檢驗 std I/O 的 handle 分別爲 0,1,2. 然後 open/close 二次。
然後 open 60 次。
故總共佔用 3+60 = 63 個 handles。



Boolan 博览

_cinit()

```
    ExitProcess(code)
    _initterm(,,)   //do terminators
      __endstdio(void)
    _initterm(,,)   //do pre-terminators
  doexit(code, 0, 0)
exit(code)
main()
  _initterm(,,) //do C++ initializations
    __initstdio(void)
  _initterm(,,) //do initializations
  _cinit()
_setenvp()
_setargv()
__crtGetEnvironmentStringsA()
GetCommandLineA()
              __sbh_alloc_new_group(...)
              __sbh_alloc_new_region()
            __sbh_alloc_block(...)
          _heap_alloc_base(...)
        _heap_alloc_dbg(...)
      _nh_malloc_dbg(...)
    _malloc_dbg(...)
  _ioinit()
    __sbh_heap_init()
  _heap_init(...)
mainCRTStartup()
```

```
/***
*_cinit - C initialization
*
*Purpose:
*       This routine performs the shared DOS and Windows initialization.
*       The following order of initialization must be preserved -
*
*       1.  Check for devices for file handles 0 - 2
*       2.  Integer divide interrupt vector setup
*       3.  General C initializer routines
*
*Entry:
*       No parameters: Called from __crtstart and assumes data
*       set up correctly there.
*
*Exit:
*       Initializes C runtime data.
*
*Exceptions:
*
*******************************************************/
```